BBM 202 - ALGORITHMS

HACETTEPE UNIVERSITY

DEPT. OF COMPUTER ENGINEERING

REGULAR EXPRESSIONS

Acknowledgement: The course slides are adapted from the slides prepared by R. Sedgewick and K. Wayne of Princeton University.

TODAY

- Regular Expressions
- REs and NFAs
- NFA simulation
- NFA construction
- Applications

Regular expressions

A regular expression is a notation to specify a set of strings.

a "language"

operation	order	example RE	matches	does not match
concatenation	3	`AABAAB`	`AABAAB`	every other string
or	4	`AA | BAAB`	`AA` `BAAB`	every other string
closure	2	`AB*A`	`AA` `ABBBBBBBBA`	`AB` `ABABA`
parentheses	1	`A(A|B)AAB`	`AAAAB` `ABAAB`	every other string
		`(AB)*A`	`A` `ABABABABABA`	`AA` `ABBA`

Regular expression shortcuts

Additional operations are often added for convenience.

operation	example RE	matches	does not match
wildcard	`.U.U.U.`	`CUMULUS` `JUGULUM`	`SUCCUBUS` `TUMULTUOUS`
character class	`[A-Za-z][a-z]*`	`word` `Capitalized`	`camelCase` `4illegal`
at least 1	`A(BC)+DE`	`ABCDE` `ABCBCDE`	`ADE` `BCDE`
exactly k	`[0-9]{5}-[0-9]{4}`	`08540-1321` `19072-5541`	`111111111` `166-54-111`
complement	`[^AEIOU]{6}`	`RHYTHM`	`DECADE`

Ex. `[A-E]+` is shorthand for `(A|B|C|D|E)(A|B|C|D|E)*`

Regular expression examples

RE notation is surprisingly expressive

regular expression	matches	does not match
`.*SPB.*` *(substring search)*	`RASPBERRY` `CRISPBREAD`	`SUBSPACE` `SUBSPECIES`
`[0-9]{3}-[0-9]{2}-[0-9]{4}` *(Social Security numbers)*	`166-11-4433` `166-45-1111`	`11-55555555` `8675309`
`[a-z]+@([a-z]+\.)+(edu|com)` *(email addresses)*	`wayne@princeton.edu` `rs@princeton.edu`	`spam@nowhere`
`[$_A-Za-z][$_A-Za-z0-9]*` *(Java identifiers)*	`ident3` `PatternMatcher`	`3a` `ident#3`

REs plays a well-understood role in the theory of computation.

Regular Expressions

- REs and NFAs
- NFA simulation
- NFA construction
- Applications

Nondeterministic finite-state automata

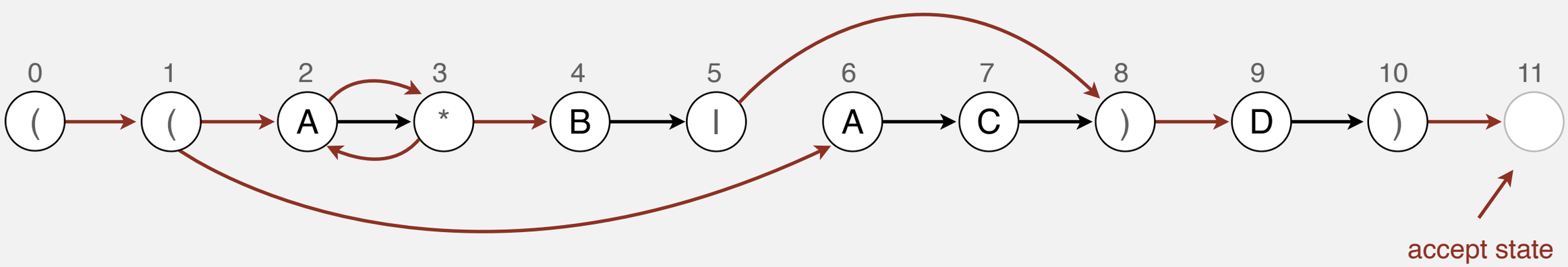
Regular-expression-matching NFA.

- RE enclosed in parentheses.
- One state per RE character (start = 0, accept = M).
- Red ε-transition (change state, but don't scan text).
- Black match transition (change state and scan to next text char).
- Accept if any sequence of transitions ends in accept state.

after scanning all text characters

Nondeterminism.

- One view: machine can guess the proper sequence of state transitions.
- Another view: sequence is a proof that the machine accepts the text.

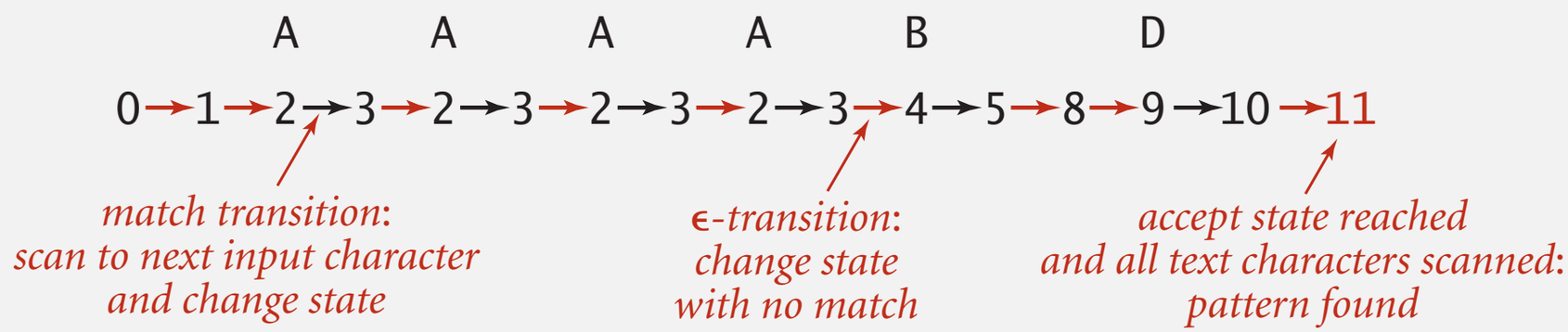


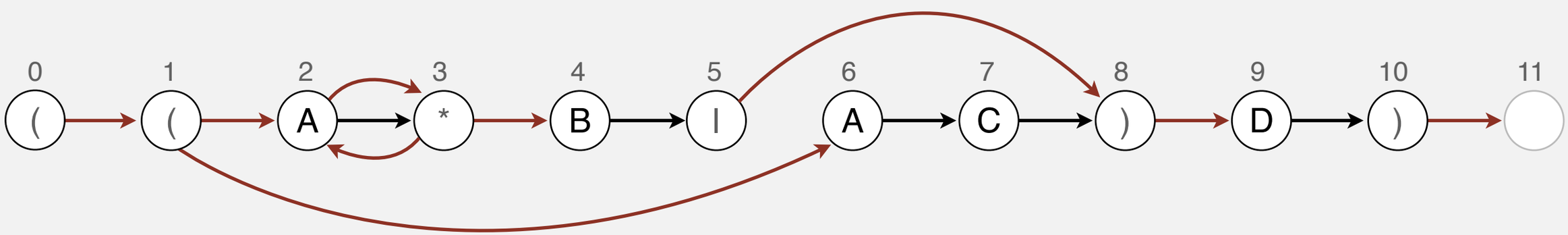
NFA corresponding to the pattern ((A * B | A C) D)

Nondeterministic finite-state automata

Q. Is AAAABD matched by NFA?

A. Yes, because some sequence of legal transitions ends in state 11.





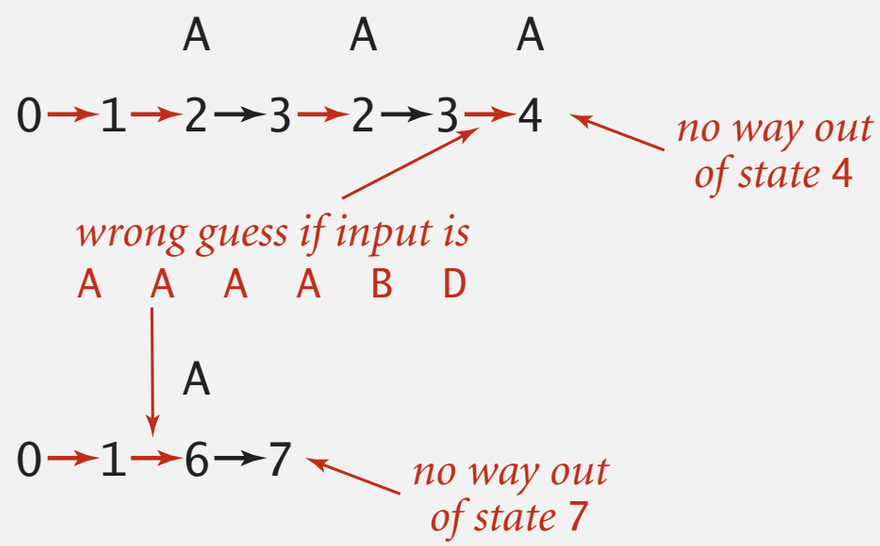
NFA corresponding to the pattern ((A * B | A C) D)

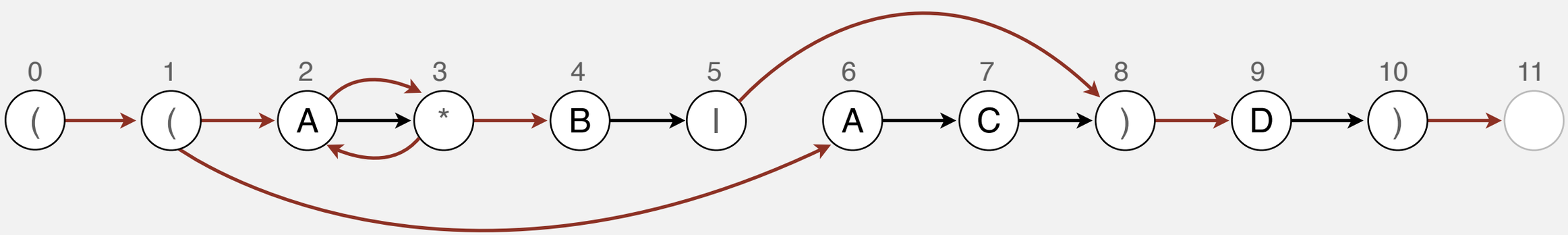
Nondeterministic finite-state automata

Q. Is AAAABD matched by NFA?

A. Yes, because some sequence of legal transitions ends in state 11.

[even though some sequences end in wrong state or stall]





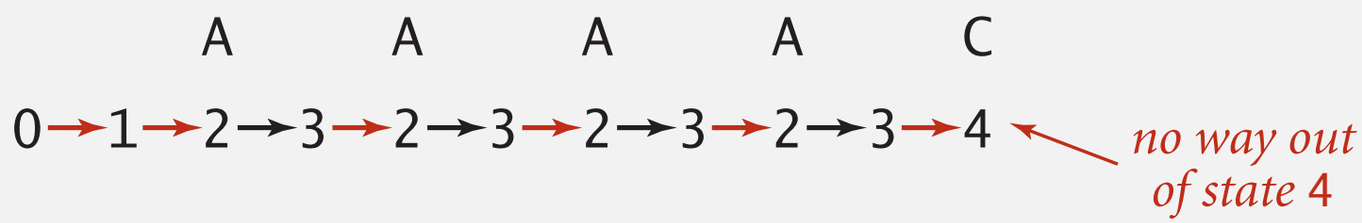
NFA corresponding to the pattern ((A * B | A C) D)

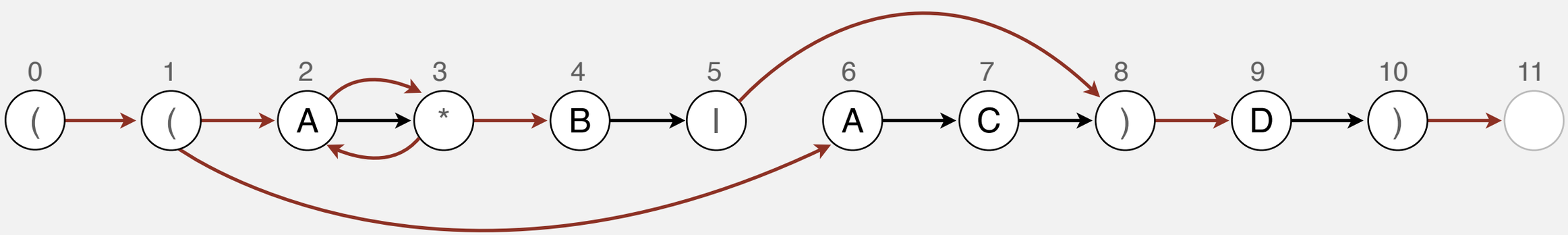
Nondeterministic finite-state automata

Q. Is AAAC matched by NFA?

A. No, because no sequence of legal transitions ends in state 11.

[but need to argue about all possible sequences]





NFA corresponding to the pattern ((A * B | A C) D)

Nondeterminism

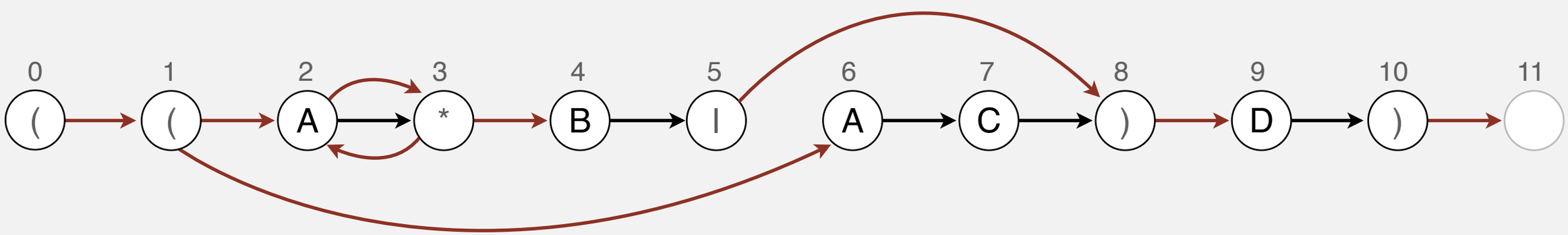
Q. How to determine whether a string is matched by an automaton?

DFA. Deterministic $\Rightarrow$ exactly one applicable transition.

NFA. Nondeterministic $\Rightarrow$ can be several applicable transitions; need to select the right one!

Q. How to simulate NFA?

A. Systematically consider all possible transition sequences.



NFA corresponding to the pattern ((A * B | A C) D)

Regular Expressions

- REs and NFAs
- **NFA simulation**
- NFA construction
- Applications

NFA representation

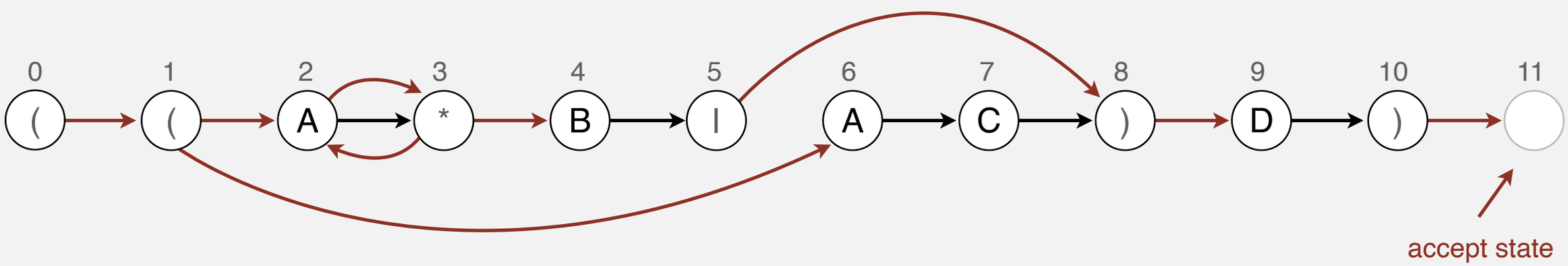
State names. Integers from 0 to M.

number of symbols in RE

Match-transitions. Keep regular expression in array `re[]`.

ε-transitions. Store in a digraph G.

- `0→1, 1→2, 1→6, 2→3, 3→2, 3→4, 5→8, 8→9, 10→11`

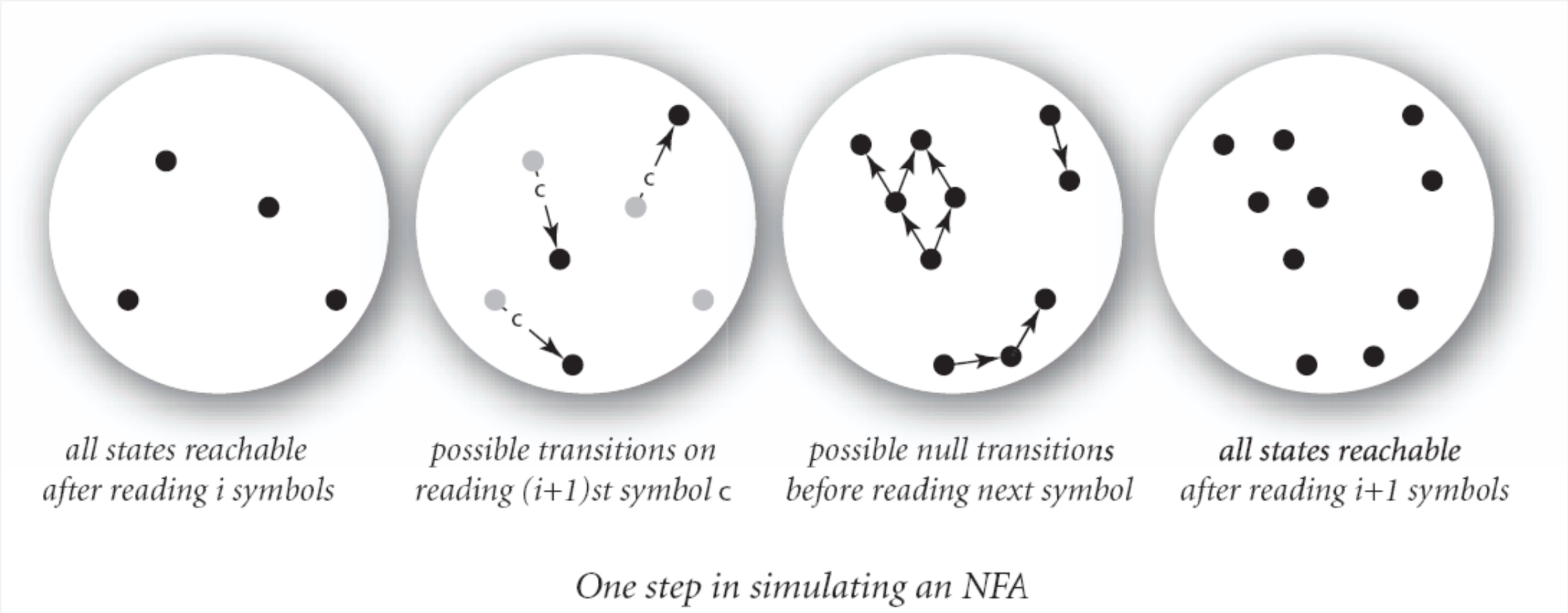


NFA corresponding to the pattern ((A * B | A C) D)

NFA simulation

Q. How to efficiently simulate an NFA?

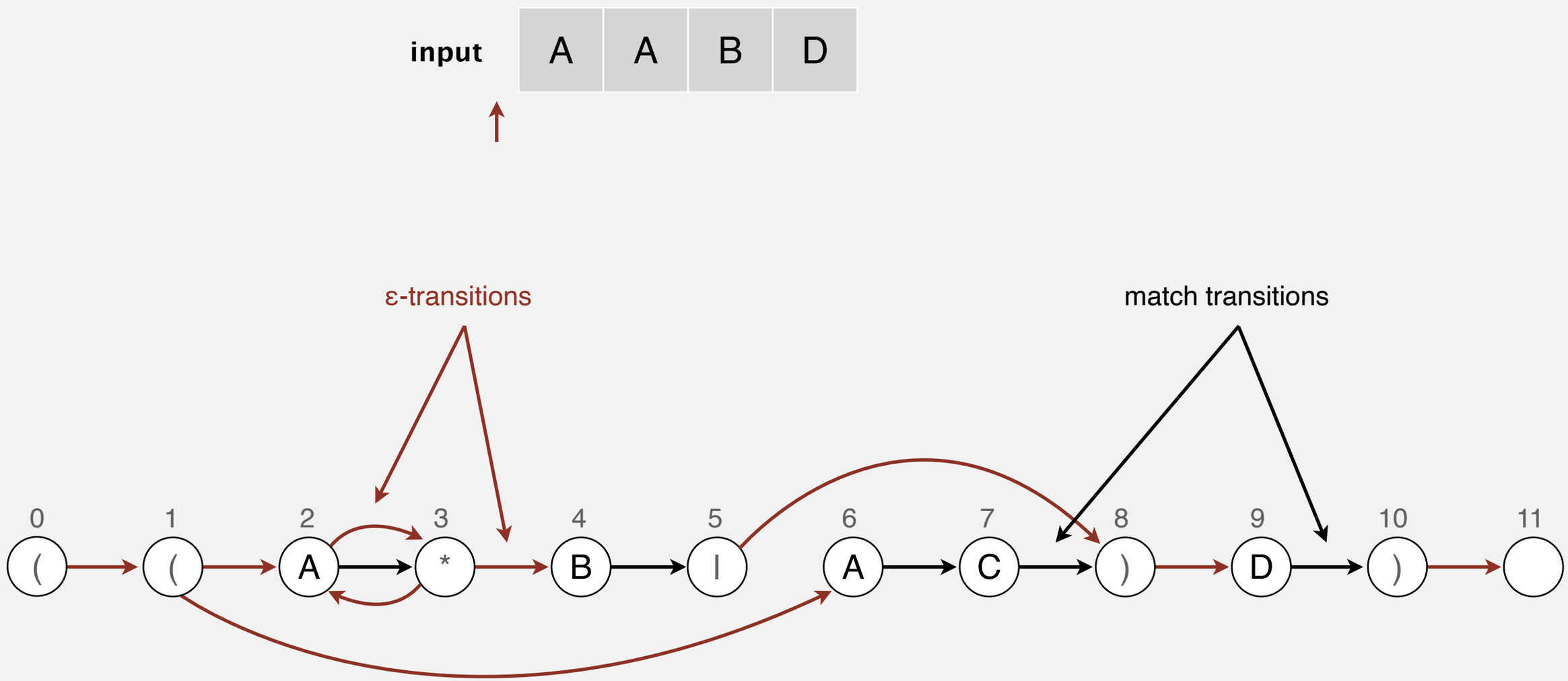
A. Maintain set of all possible states that NFA could be in after reading in the first i text characters.



Q. How to perform reachability?

NFA simulation

Goal. Check whether input matches pattern.

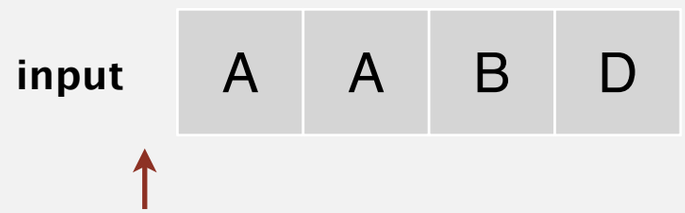


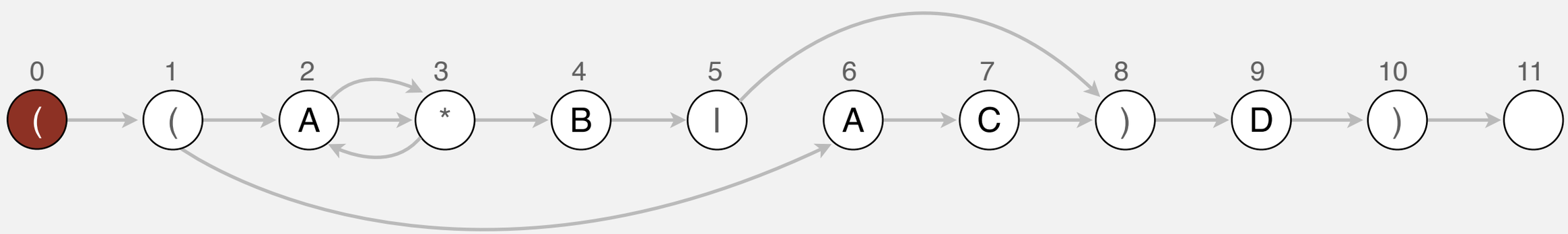
NFA corresponding to the pattern ((A * B | A C) D)

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions



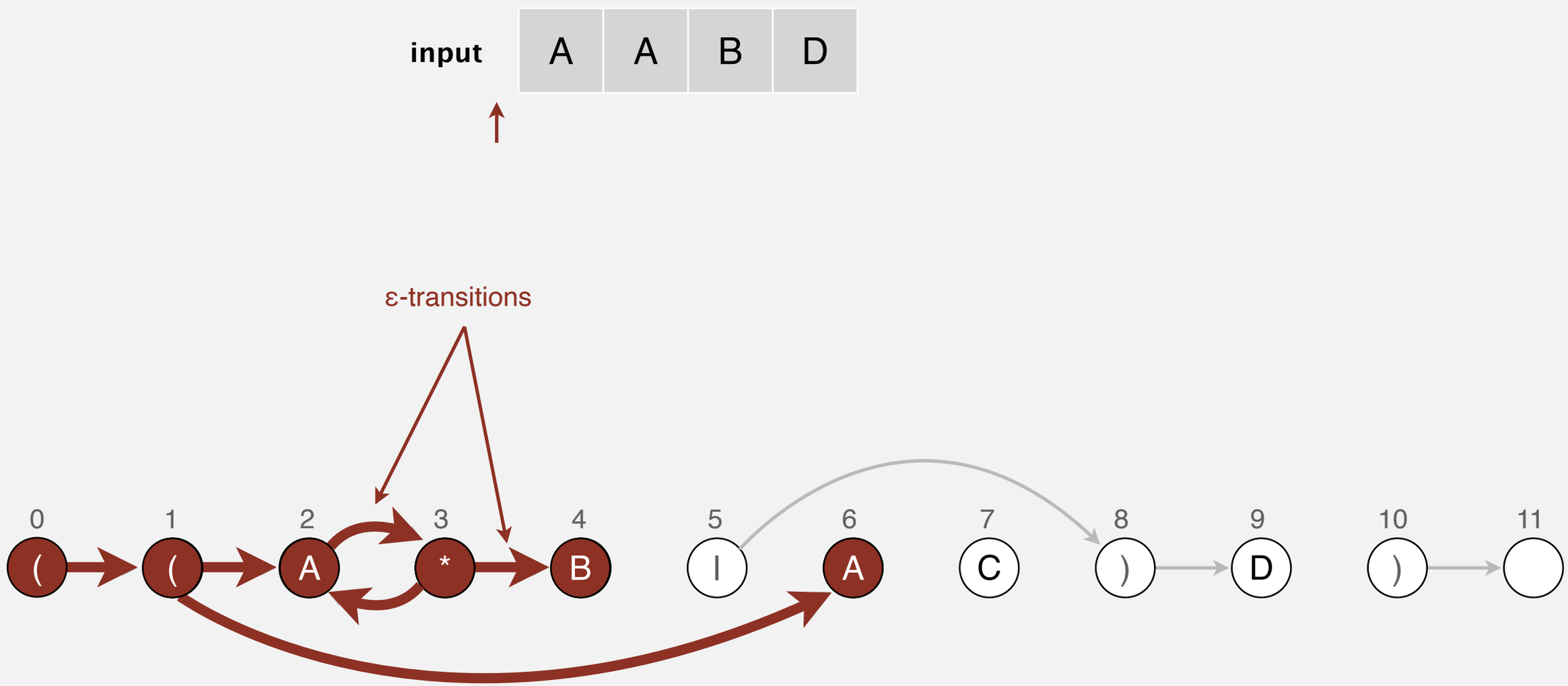


set of states reachable from start: 0

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

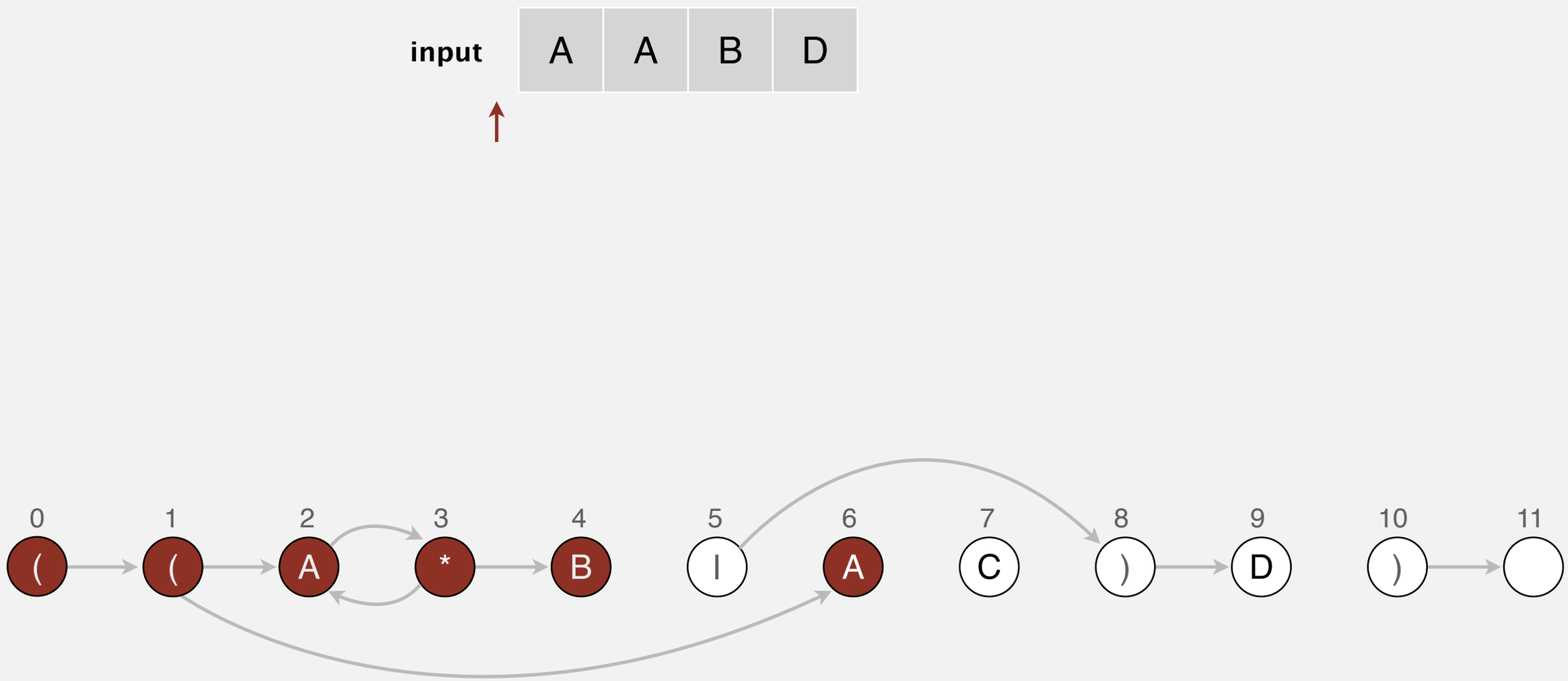


set of states reachable via ε-transitions from start

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

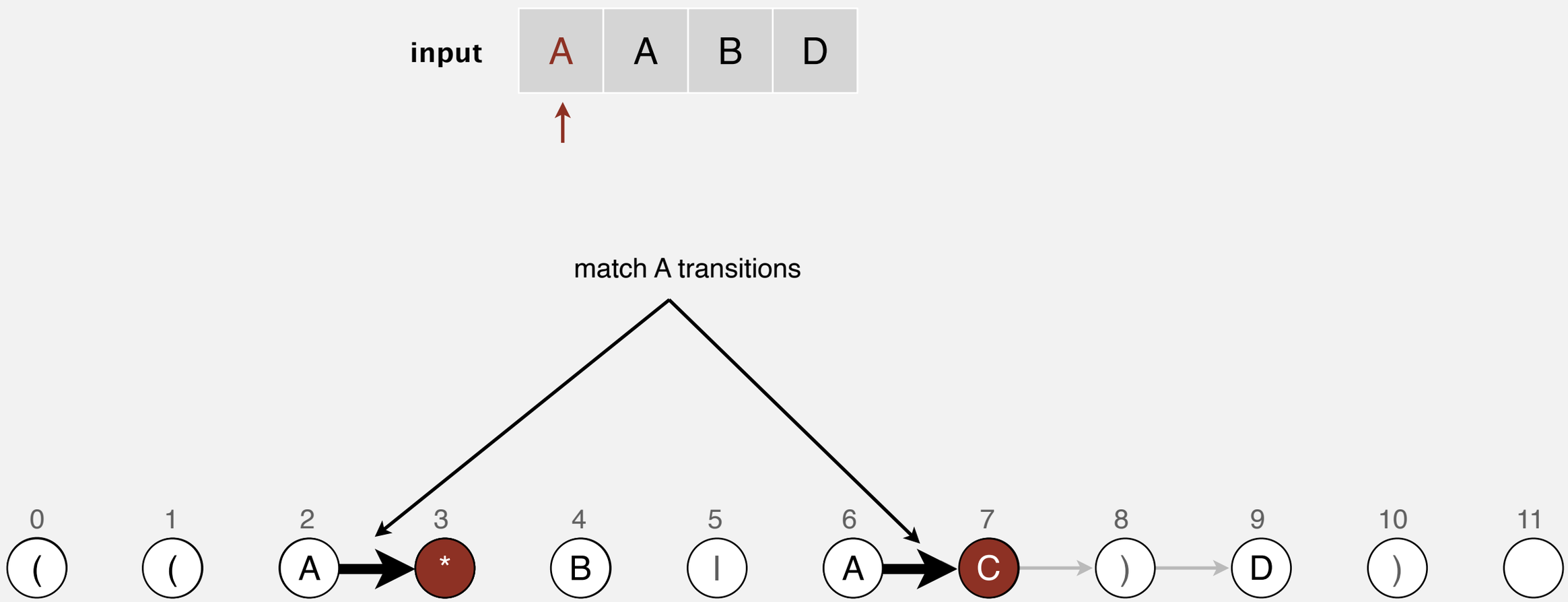


set of states reachable via ε-transitions from start : { 0, 1, 2, 3, 4, 6 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

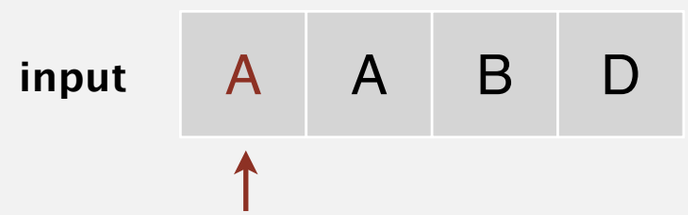


set of states reachable after matching A

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

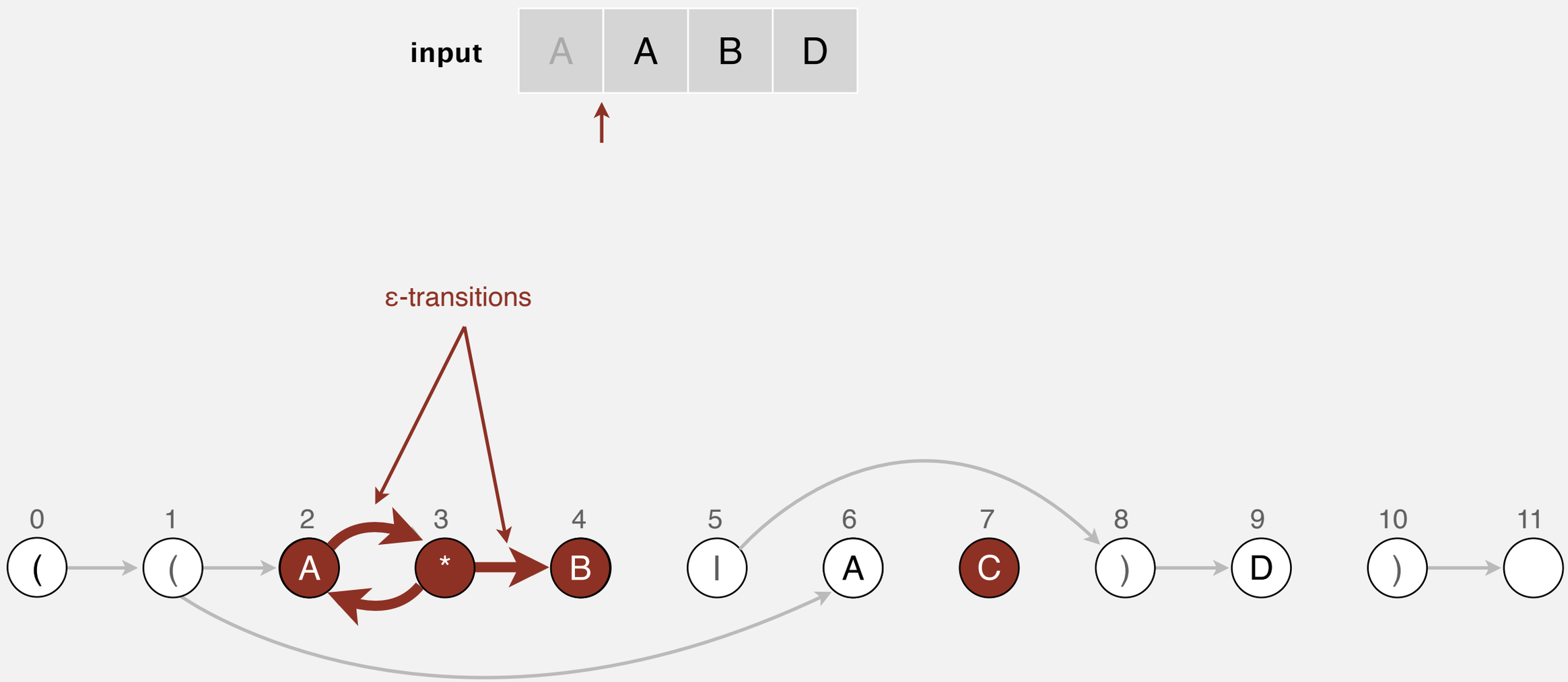


set of states reachable after matching A : { 3, 7 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

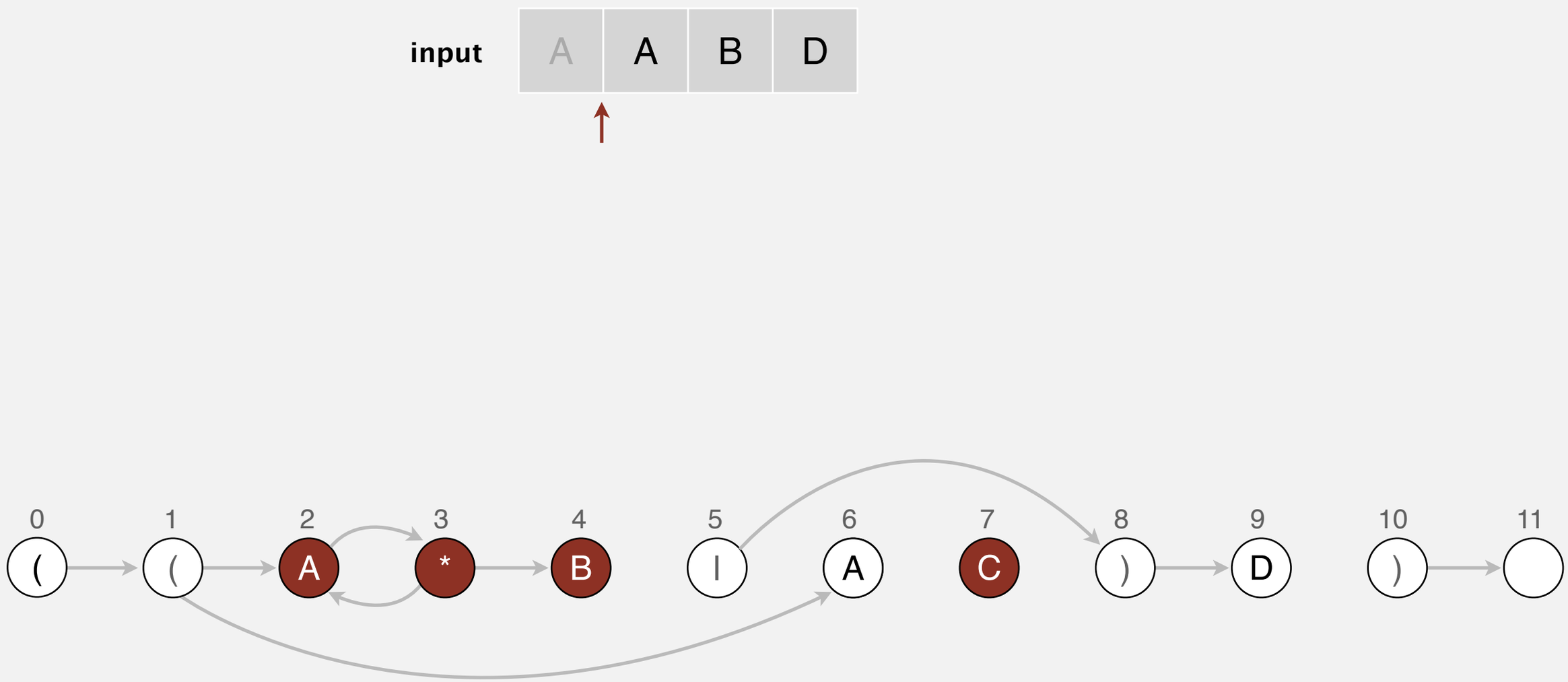


set of states reachable via ε-transitions after matching A

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

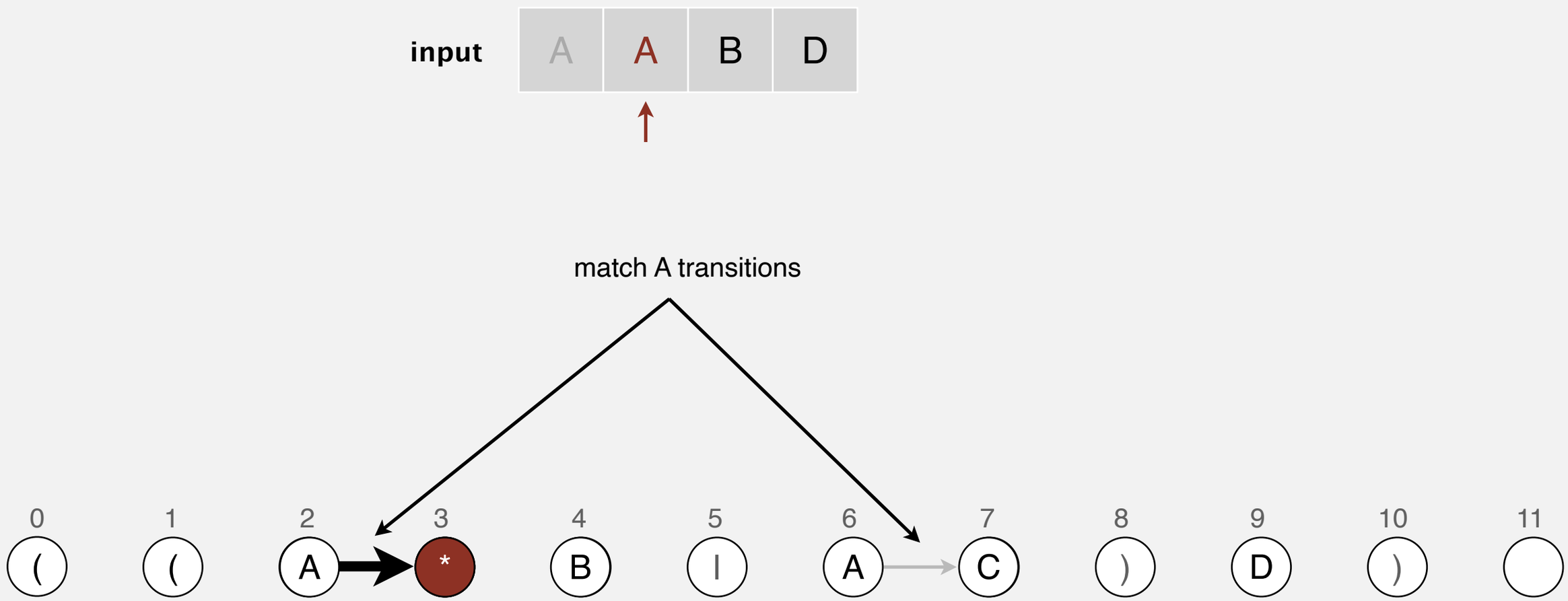


set of states reachable via ε-transitions after matching A : { 2, 3, 4, 7 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

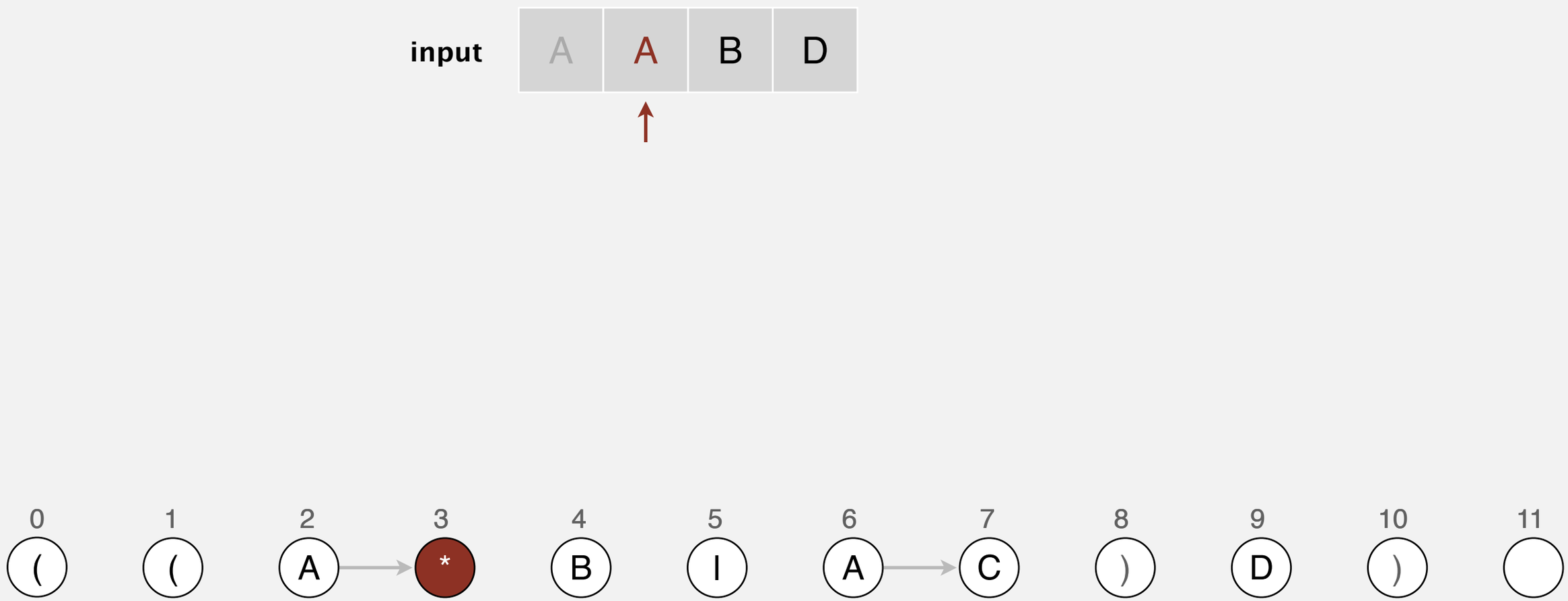


set of states reachable after matching A A

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

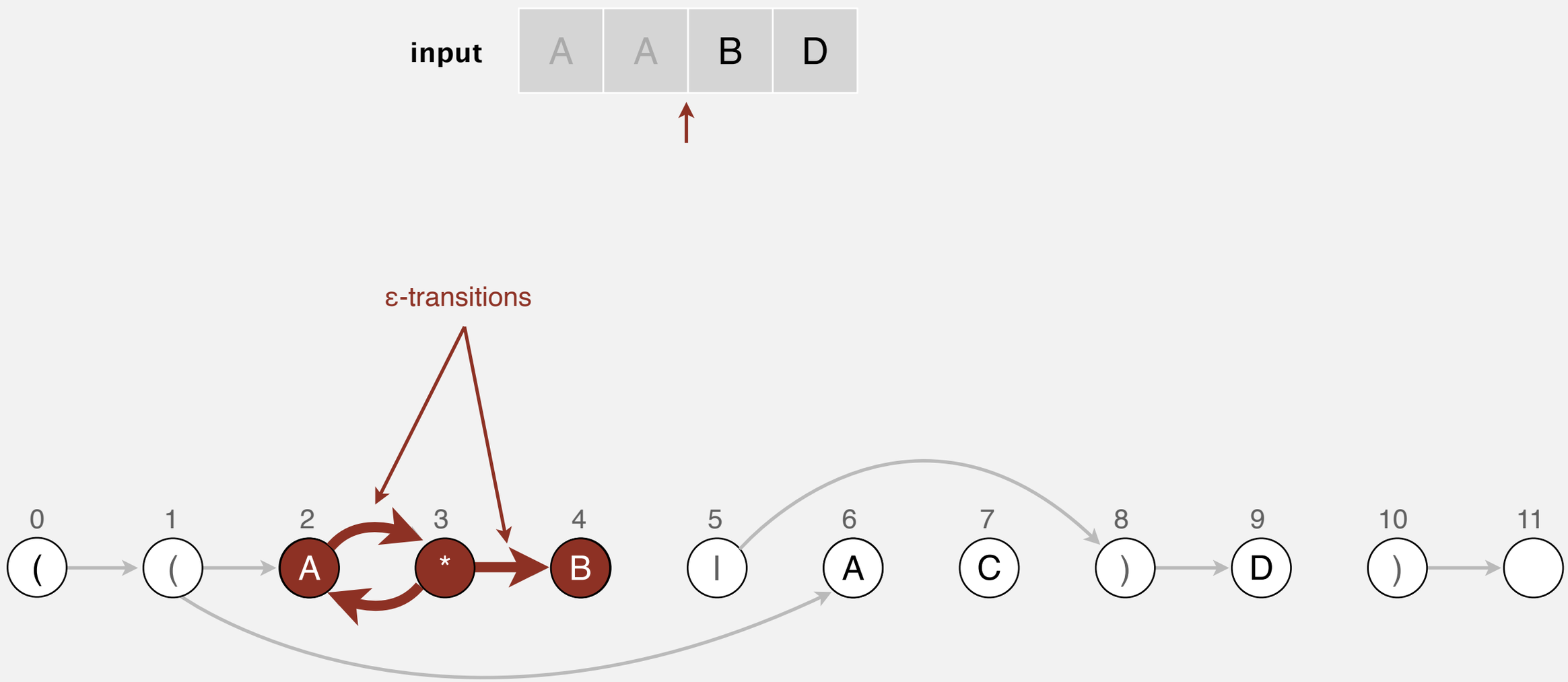


set of states reachable after matching A A : { 3 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

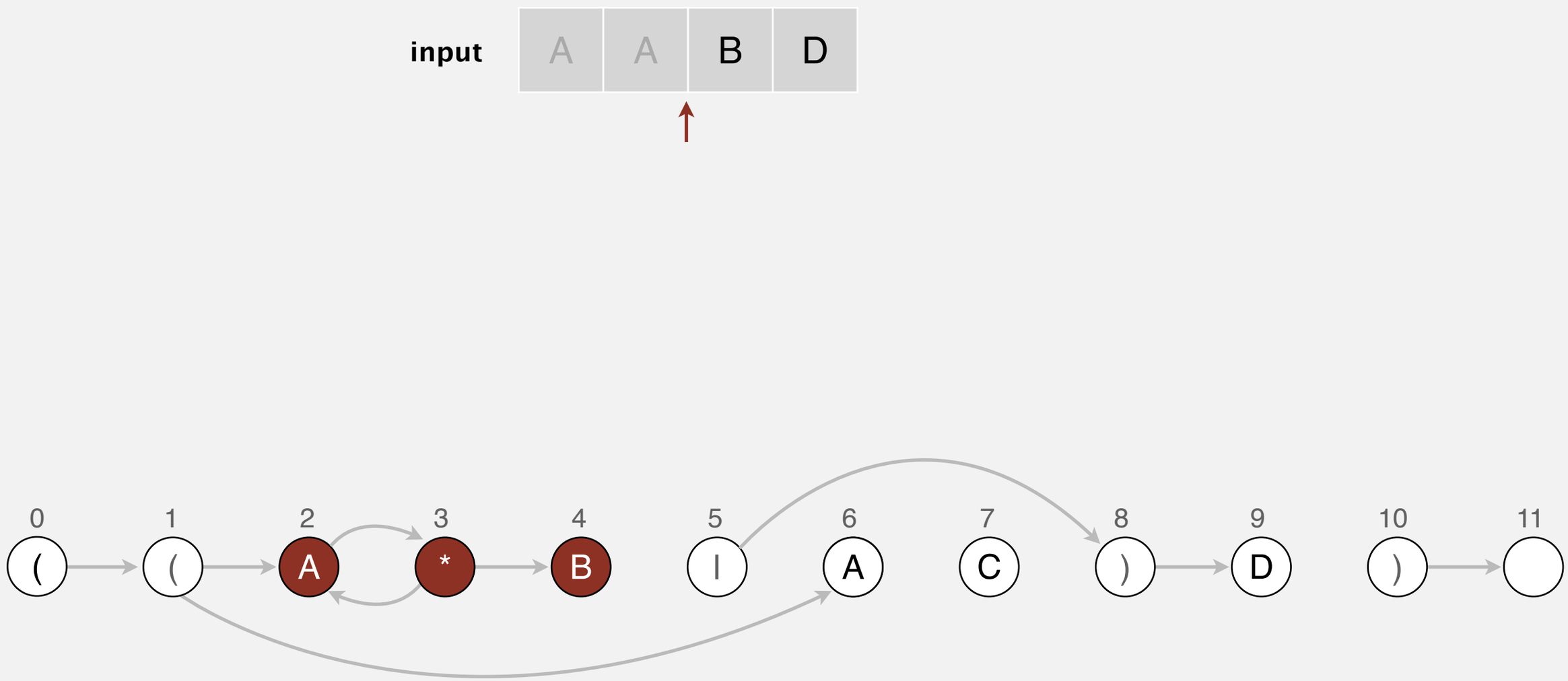


set of states reachable via ε-transitions after matching A A

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

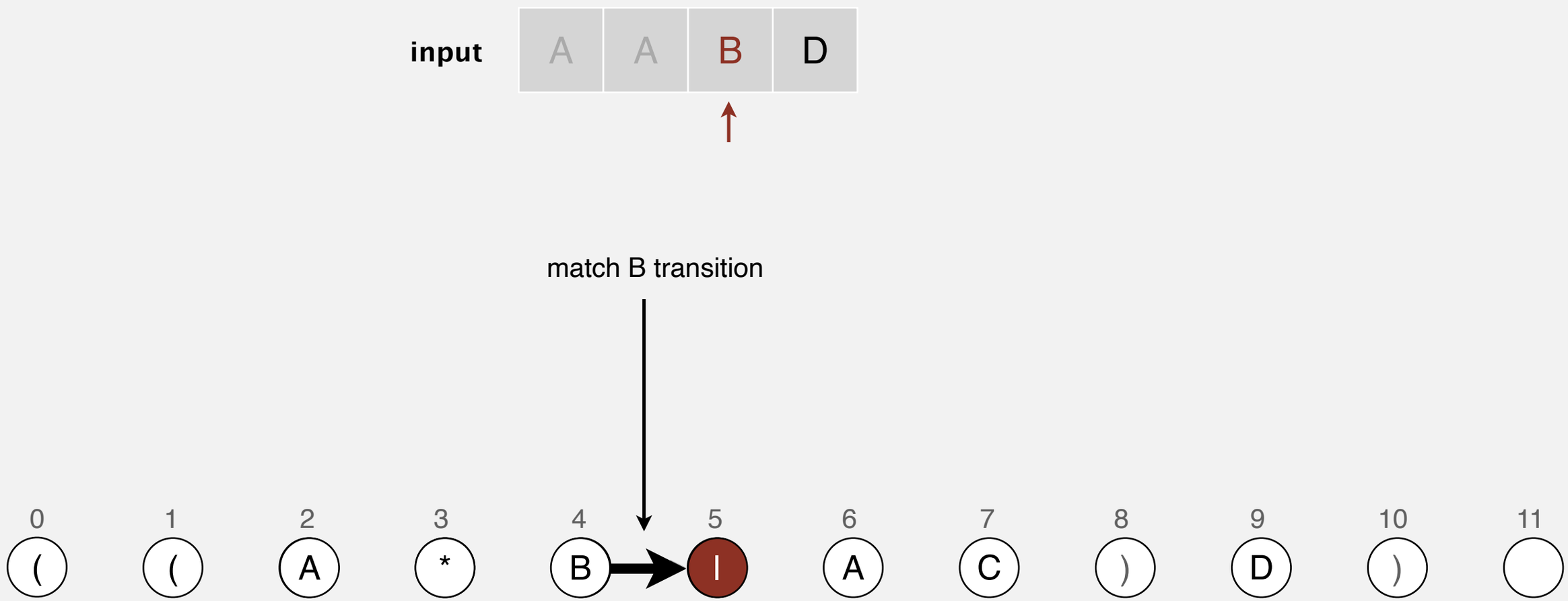


set of states reachable via ε-transitions after matching A A : { 2, 3, 4 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

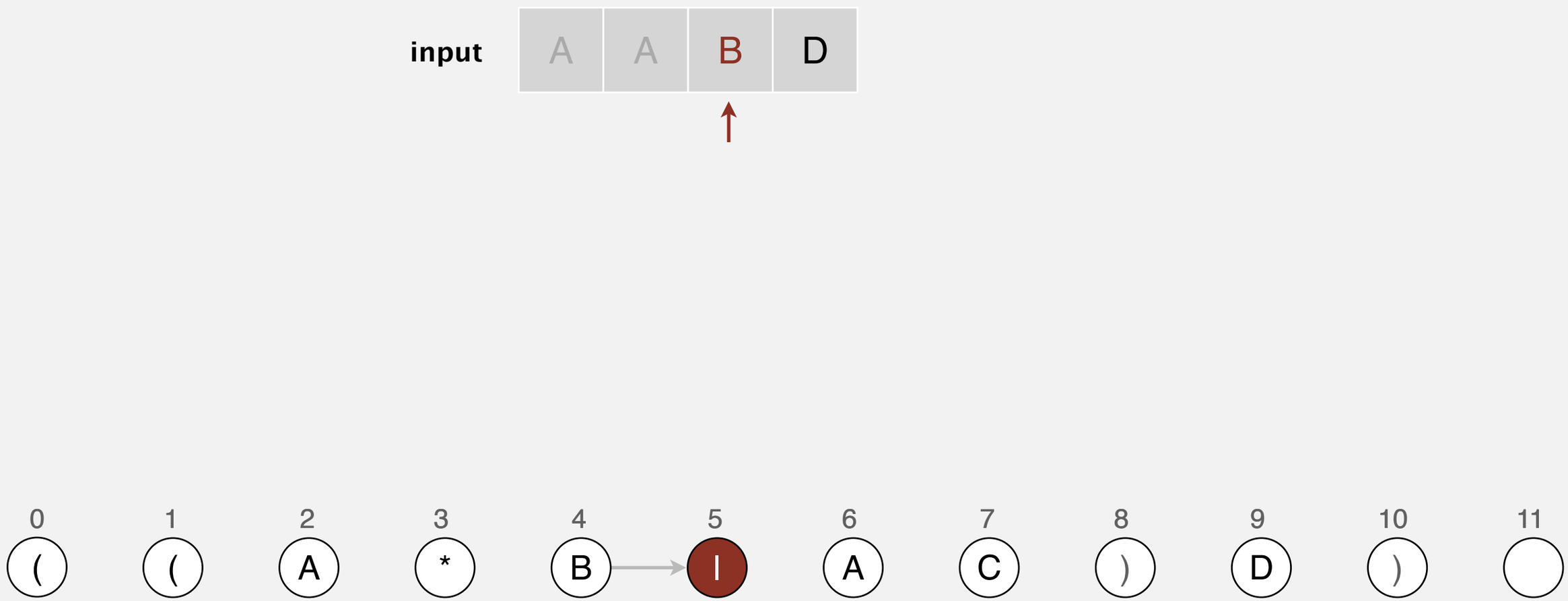


set of states reachable after matching A A B

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

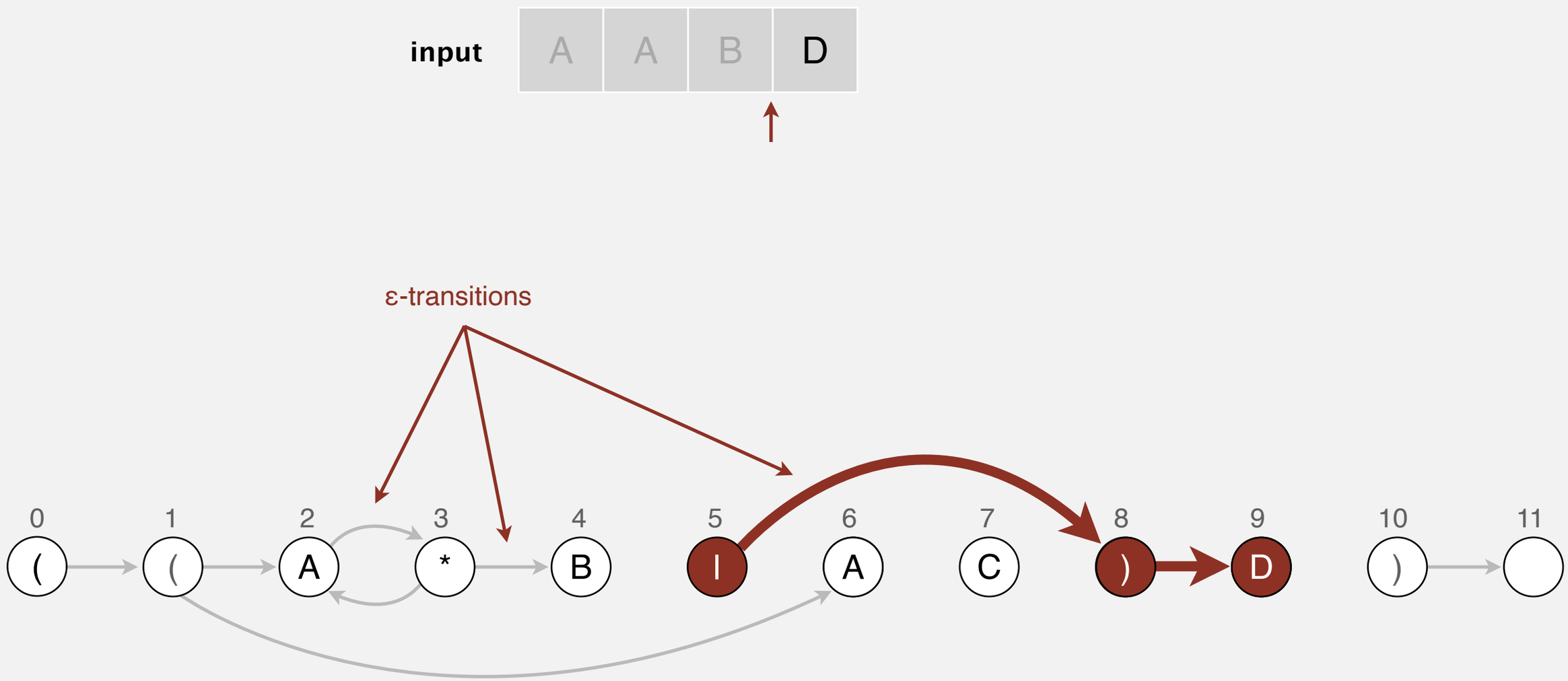


set of states reachable after matching A A B : { 5 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

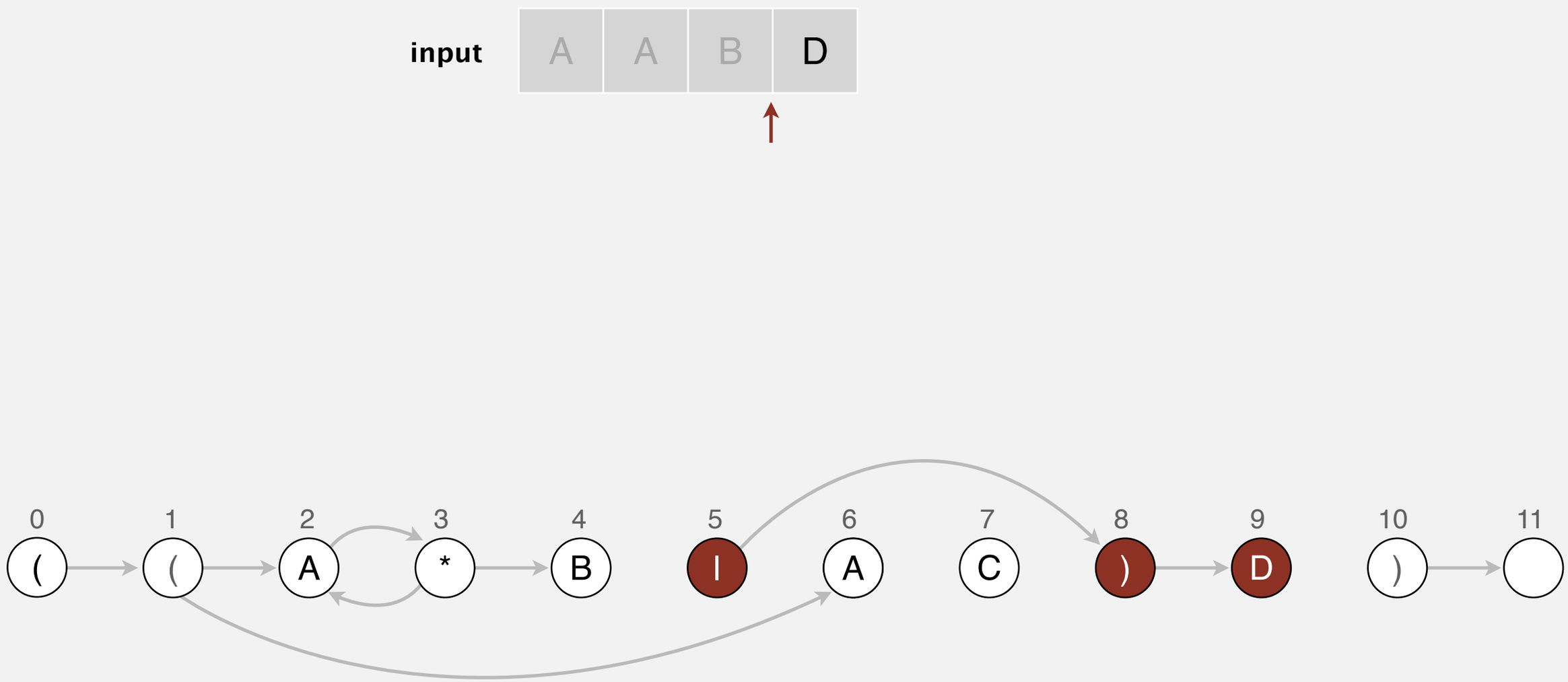


set of states reachable via ε-transitions after matching A A B

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

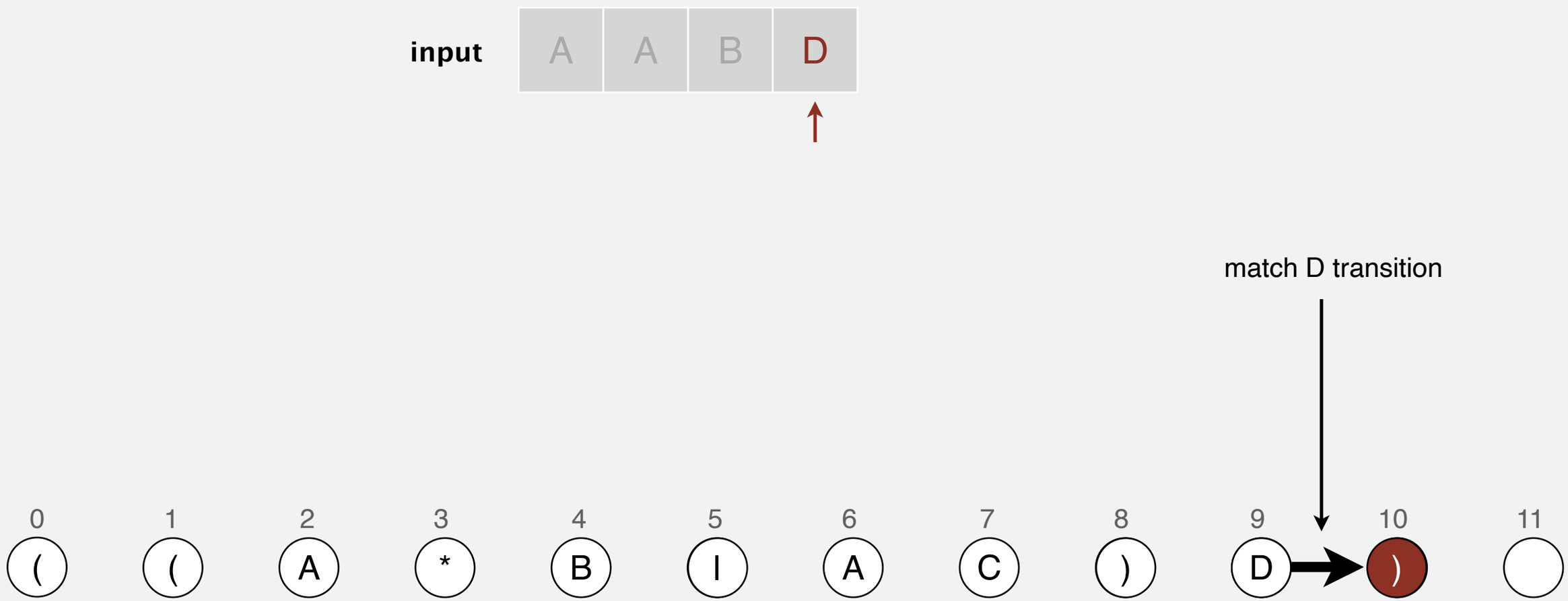


set of states reachable via ε-transitions after matching A A B : { 5, 8, 9 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

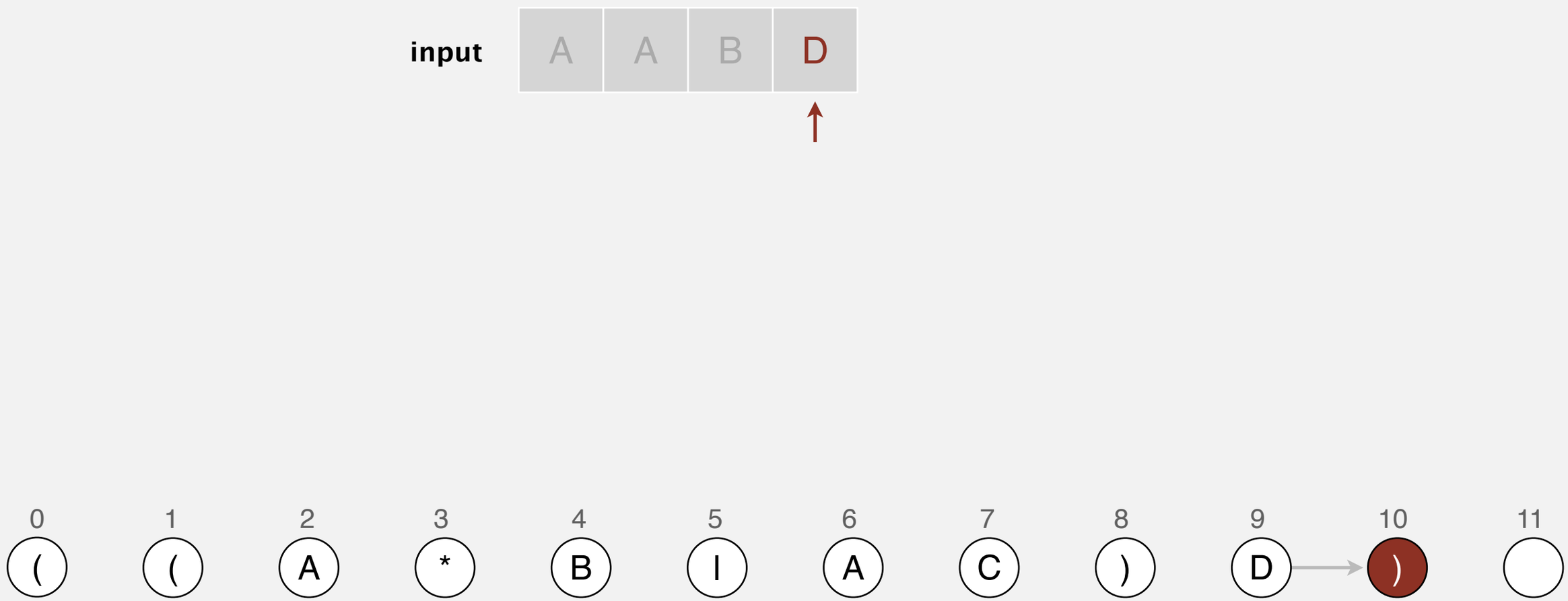


set of states reachable after matching A A B D

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

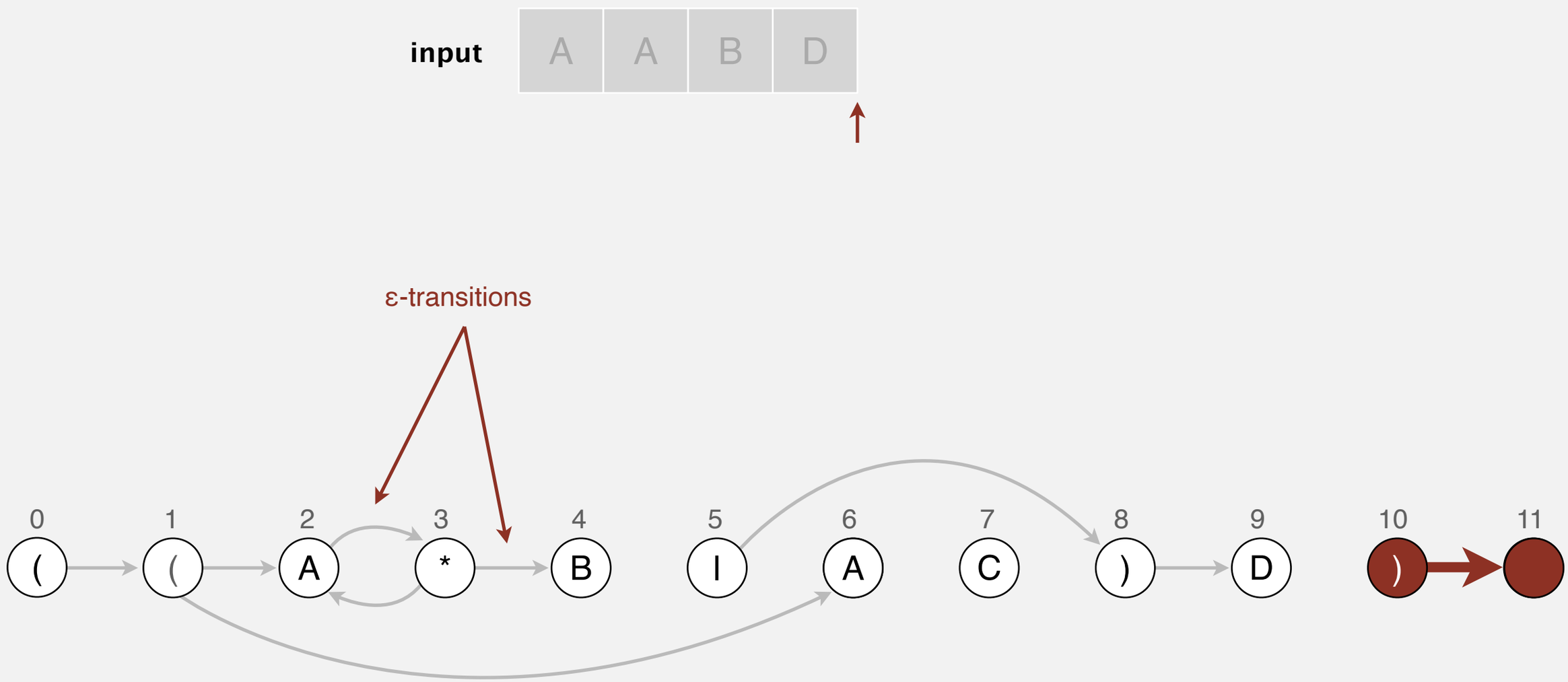


set of states reachable after matching A A B D : { 10 }

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

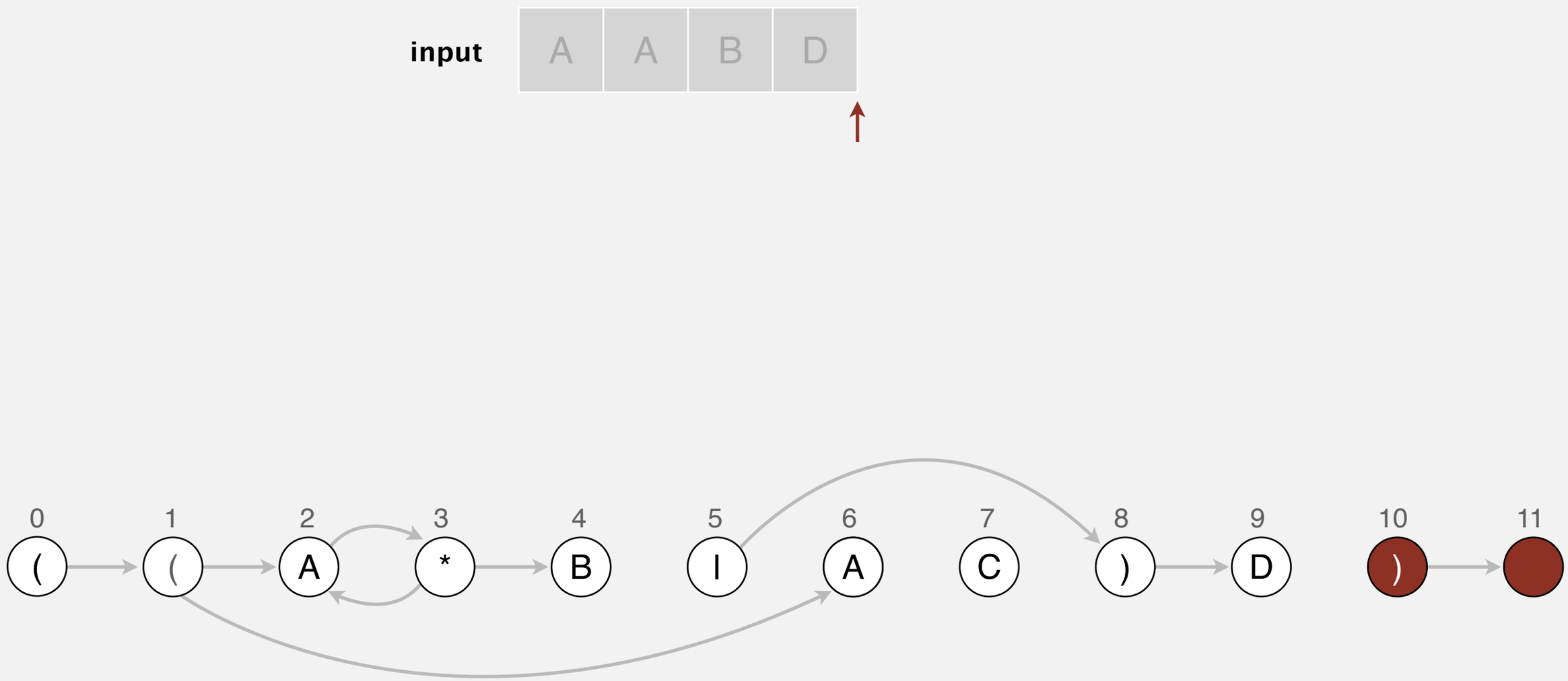


set of states reachable via ε-transitions after matching A A B D

NFA simulation

Read next input character.

- Find states reachable by match transitions.
- Find states reachable by ε-transitions

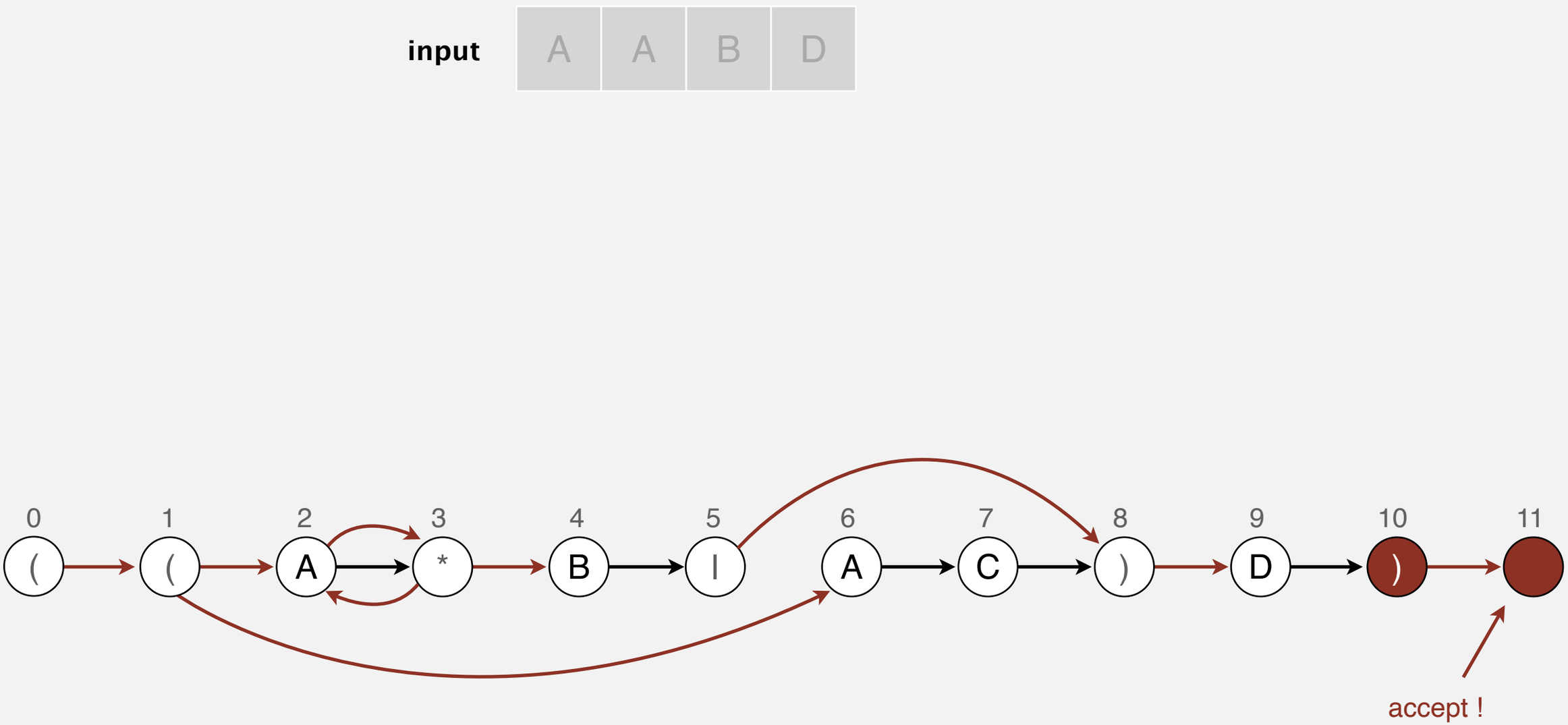


set of states reachable via ε-transitions after matching A A B D : { 10, 11 }

NFA simulation

When no more input characters:

- Accept if any state reachable is an accept state.
- Reject otherwise.



set of states reachable : { 10, 11 }

Digraph reachability

Digraph reachability. Find all vertices reachable from a given source or set of vertices.

public class DirectedDFS	
DirectedDFS(Digraph G, int s)	*find vertices reachable from s*
DirectedDFS(Digraph G, Iterable<Integer> s)	*find vertices reachable from sources*
boolean marked(int v)	*is v reachable from source(s)?*

Solution. Run DFS from each source, without unmarking vertices.

Performance. Runs in time proportional to $E + V$.

NFA simulation: Java implementation

```
public class NFA
{
   private char[] re;       // match transitions
   private Digraph G;       // epsilon transition digraph
   private int M;           // number of states

   public NFA(String regexp)
   {
      M  = regexp.length();
      re = regexp.toCharArray();
      G = buildEpsilonTransitionsDigraph();
   }

   public boolean recognizes(String txt)
   {  /* see next slide */  }

   public Digraph buildEpsilonTransitionDigraph()
   {  /* stay tuned */  }

}
```

NFA simulation: Java implementation

```
public boolean recognizes(String txt)
{
   Bag<Integer> pc = new Bag<Integer>();
   DirectedDFS dfs = new DirectedDFS(G, 0);
   for (int v = 0; v < G.V(); v++)
      if (dfs.marked(v)) pc.add(v);

   for (int i = 0; i < txt.length(); i++)
   {
      Bag<Integer> match = new Bag<Integer>();
      for (int v : pc)
      {
         if (v == M) continue;
         if ((re[v] == txt.charAt(i)) || re[v] == '.')
            match.add(v+1);
      }

      dfs = new DirectedDFS(G, match);
      pc = new Bag<Integer>();
      for (int v = 0; v < G.V(); v++)
         if (dfs.marked(v)) pc.add(v);
   }

   for (int v : pc)
      if (v == M) return true;
   return false;
}
```

states reachable from start by ε-transitions

states reachable after scanning past `txt.charAt(i)`

follow ε-transitions

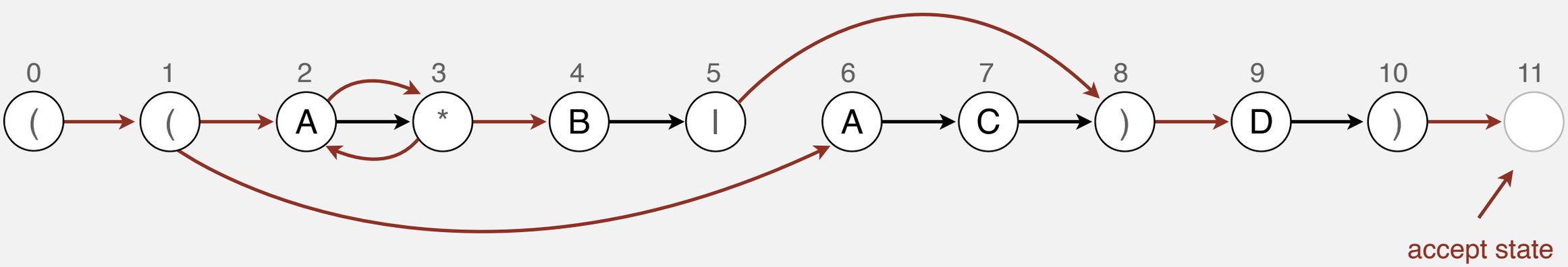
accept if can end in state M

NFA simulation: analysis

Proposition. Determining whether an N-character text is recognized by the NFA corresponding to an M-character pattern takes time proportional to $M\,N$ in the worst case.

Pf. For each of the N text characters, we iterate through a set of states of size no more than M and run DFS on the graph of ε-transitions.

[The NFA construction we will consider ensures the number of edges ≤ $3M$.]



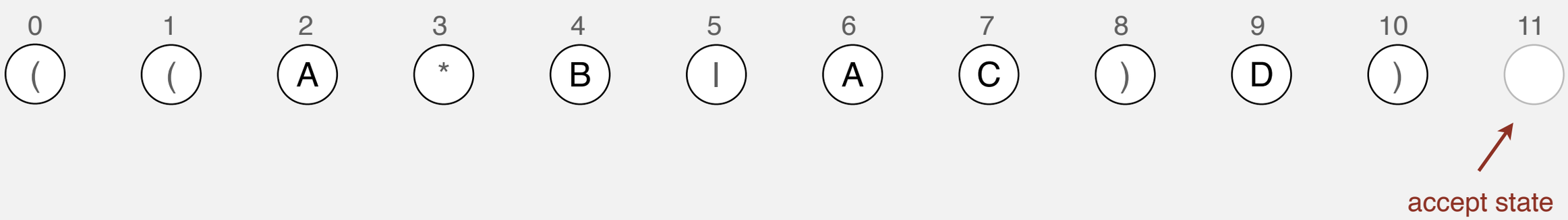
NFA corresponding to the pattern ((A * B | A C) D)

Regular Expressions

- REs and NFAs
- NFA simulation
- **NFA construction**
- Applications

Building an NFA corresponding to an RE

States. Include a state for each symbol in the RE, plus an accept state.



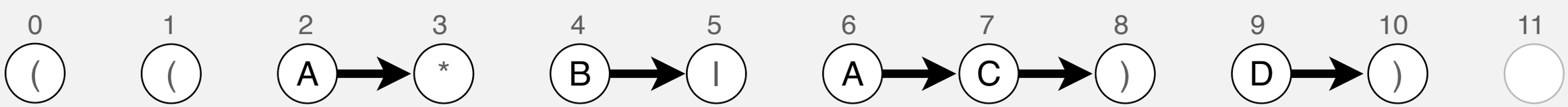
NFA corresponding to the pattern ((A * B | A C) D)

Building an NFA corresponding to an RE

Concatenation. Add match-transition edge from state corresponding to characters in the alphabet to next state.

Alphabet. `A B C D`

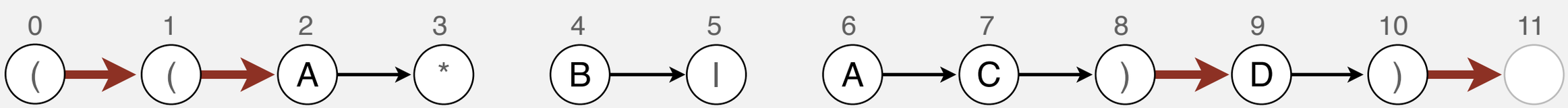
Metacharacters. `( ) . * |`



NFA corresponding to the pattern ((A * B | A C) D)

Building an NFA corresponding to an RE

Parentheses. Add ε-transition edge from parentheses to next state.

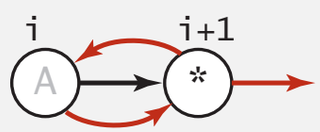


NFA corresponding to the pattern ((A * B | A C) D)

Building an NFA corresponding to an RE

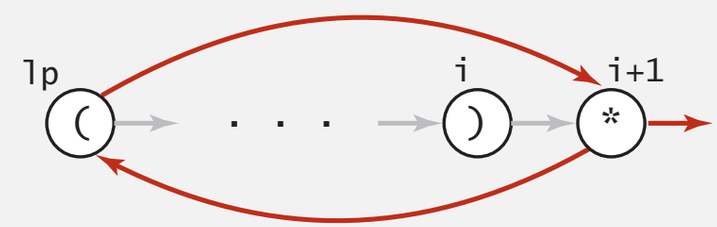
Closure. Add three ε-transition edges for each * operator.

single-character closure




```
G.addEdge(i, i+1);
G.addEdge(i+1, i);
```

closure expression

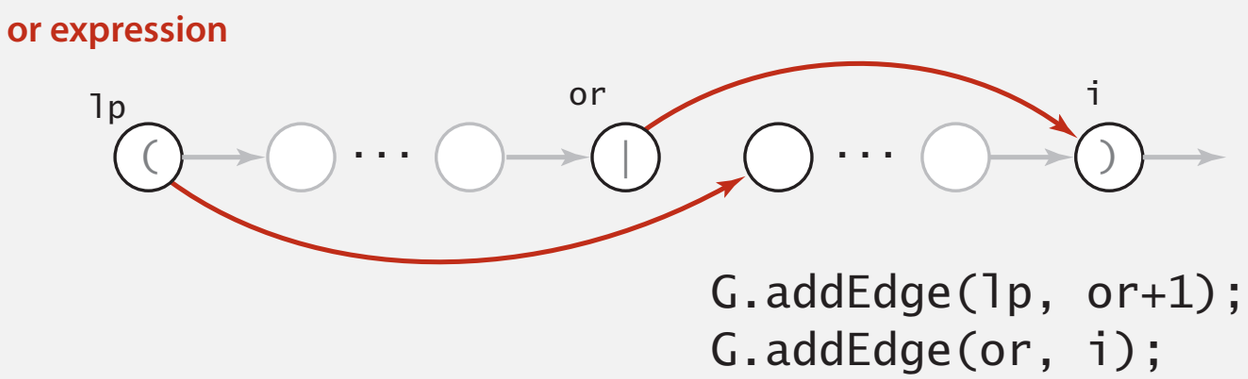


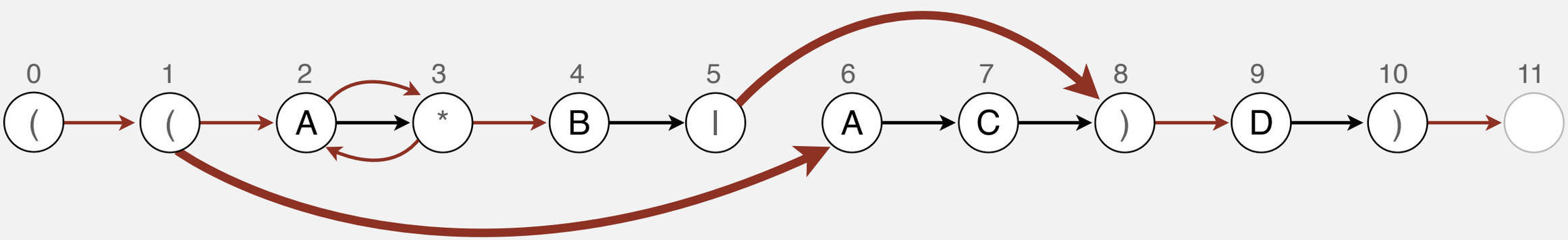

```
G.addEdge(lp, i+1);
G.addEdge(i+1, lp);
```

NFA corresponding to the pattern ((A * B | A C) D)

Building an NFA corresponding to an RE

Or. Add two ε-transition edges for each | operator.





NFA corresponding to the pattern ((A * B | A C) D)

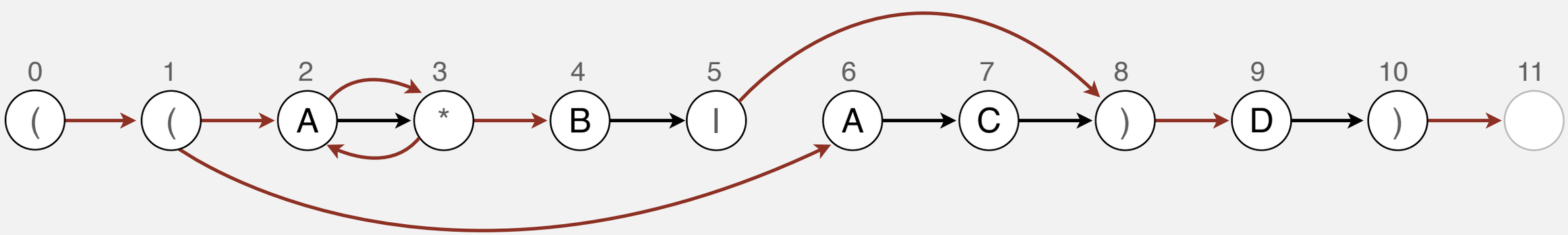
NFA construction: implementation

Goal. Write a program to build the ε-transition digraph.

Challenges. Remember left parentheses to implement closure and or; need to remember | to implement or.

Solution. Maintain a stack.

- (symbol: push (onto stack.
- | symbol: push | onto stack.
-) symbol: pop corresponding (and possibly intervening |; add ε-transition edges for closure/or.



NFA corresponding to the pattern ((A * B | A C) D)

NFA construction

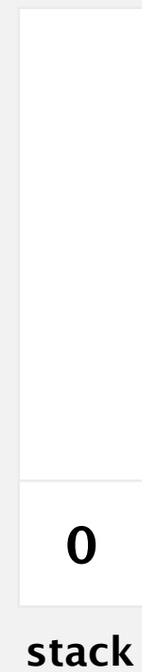
stack

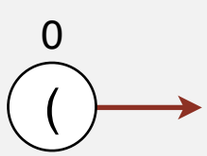
((A * B | A C) D)

NFA construction

Left parenthesis.

- Add ε-transition to next state.
- Push index of state corresponding to (onto stack.



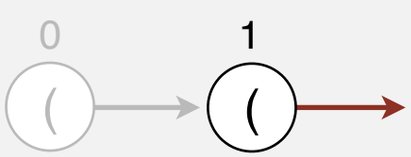


((A * B | A C) D)

NFA construction

Left parenthesis.

- Add ε-transition to next state.
- Push index of state corresponding to (onto stack.

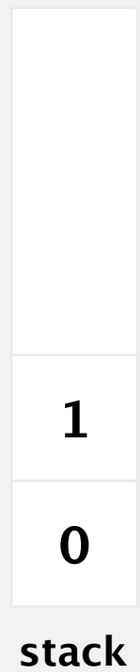


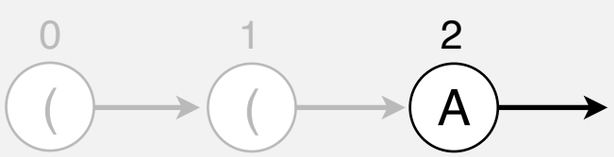
((A * B | A C) D)

NFA construction

Alphabet symbol.

- Add match transition to next state.
- Do one-character lookahead:
  add ε-transitions if next character is *.



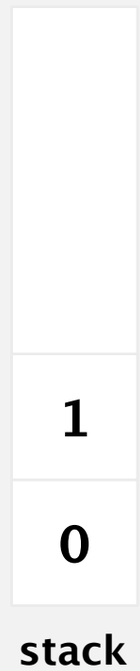


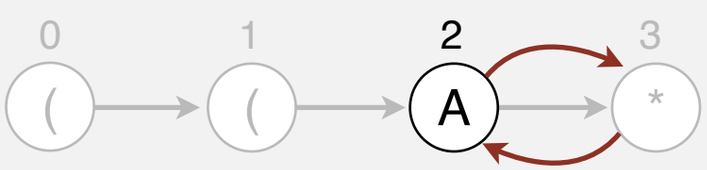
((A * B | A C) D)

NFA construction

Alphabet symbol.

- Add match transition to next state.
- Do one-character lookahead:
  add ε-transitions if next character is *.



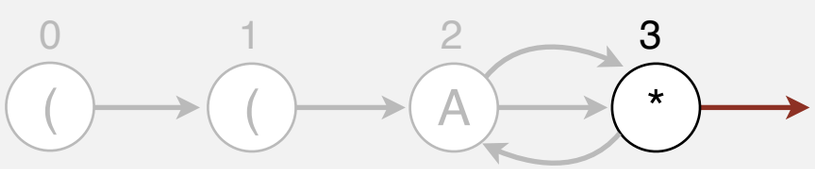


((A * B | A C) D)

NFA construction

Closure symbol.

- Add ε-transition to next state.

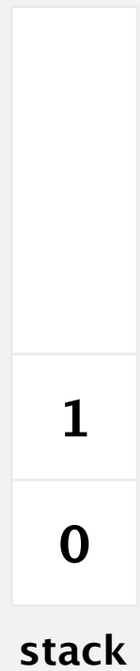


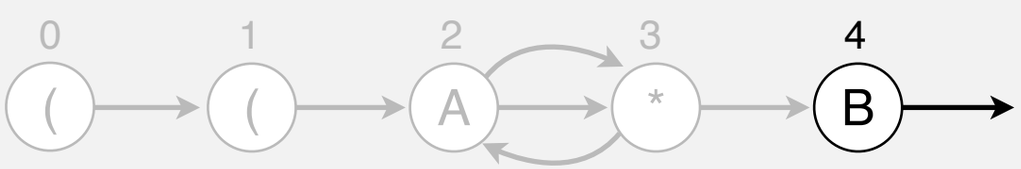
((A * B | A C) D)

NFA construction

Alphabet symbol.

- Add match transition to next state.
- Do one-character lookahead:
  add ε-transitions if next character is *.



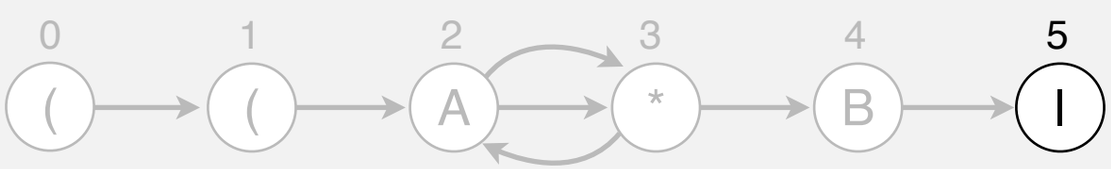


((A * B | A C) D)

NFA construction

Or symbol.

- Push index of state corresponding to | onto stack.

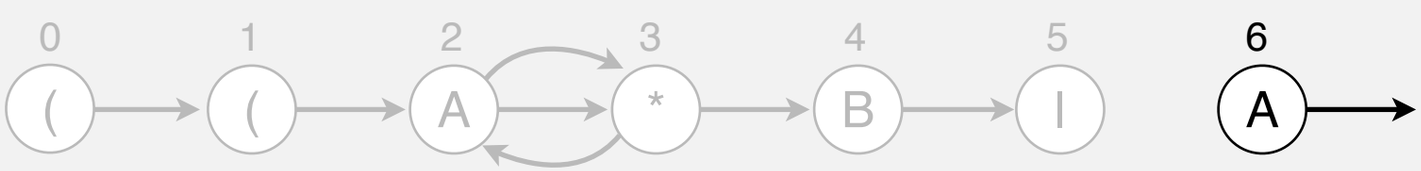


((A * B | A C) D)

NFA construction

Alphabet symbol.

- Add match transition to next state.
- Do one-character lookahead:
  add ε-transitions if next character is *.

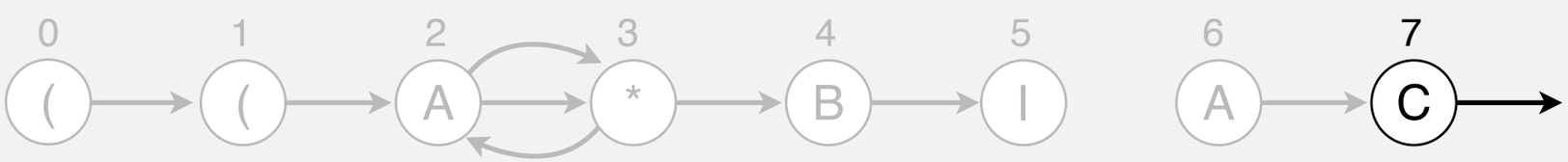


((A * B | A C) D)

NFA construction

Alphabet symbol.

- Add match transition to next state.
- Do one-character lookahead:
  add ε-transitions if next character is *.

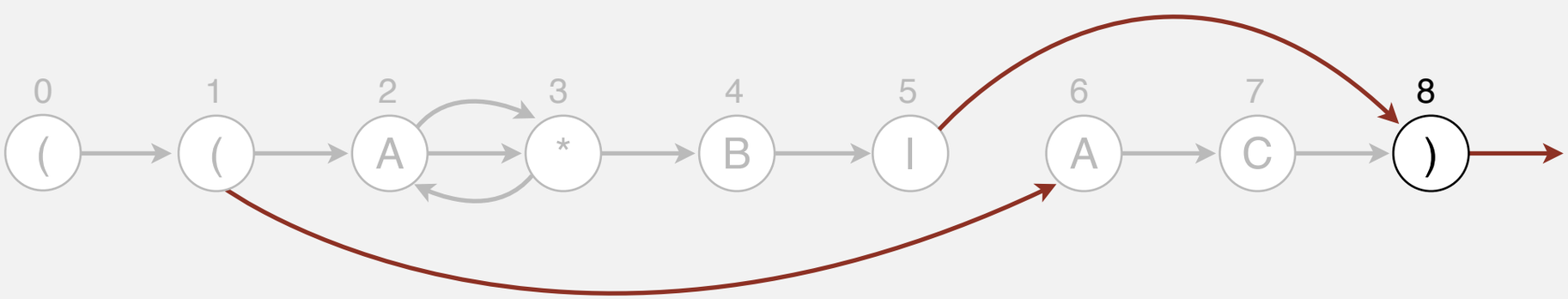


((A * B | A C) D)

NFA construction

Right parenthesis.

- Add ε-transition to next state.
- Pop corresponding (and possibly intervening |; add ε-transition edges for or.
- Do one-character lookahead: add ε-transitions if next character is *.

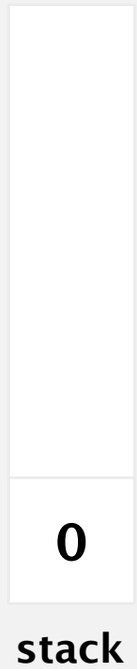


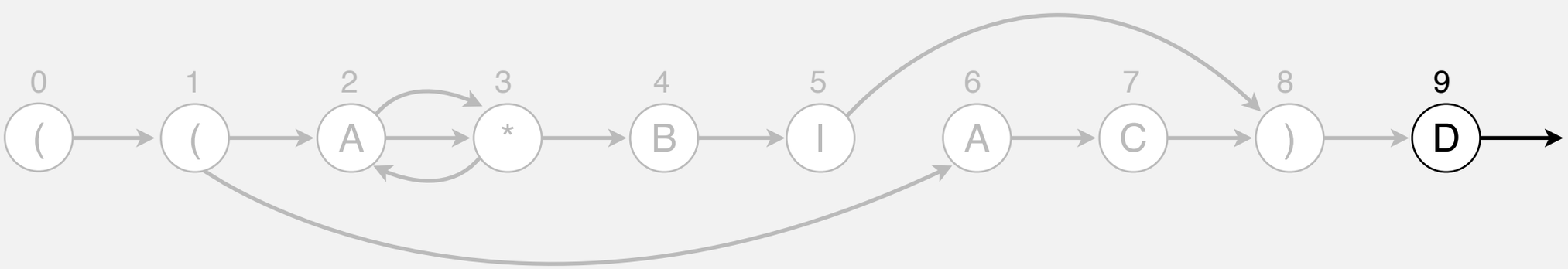
((A * B | A C) D)

NFA construction

Alphabet symbol.

- Add match transition to next state.
- Do one-character lookahead:
  add ε-transitions if next character is *.



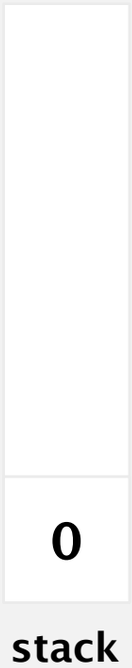


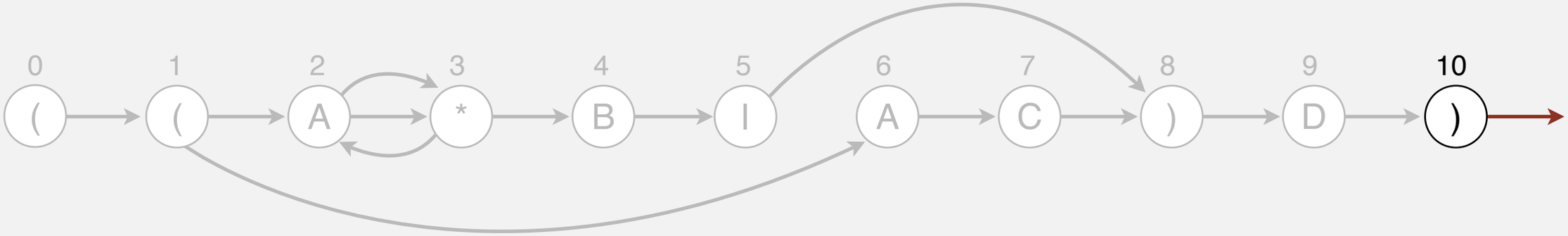
((A * B | A C) D)

NFA construction

Right parenthesis.

- Add ε-transition to next state.
- Pop corresponding (and possibly intervening |; add ε-transition edges for or.
- Do one-character lookahead: add ε-transitions if next character is *.



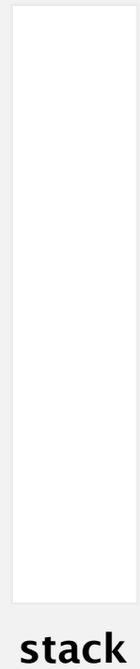


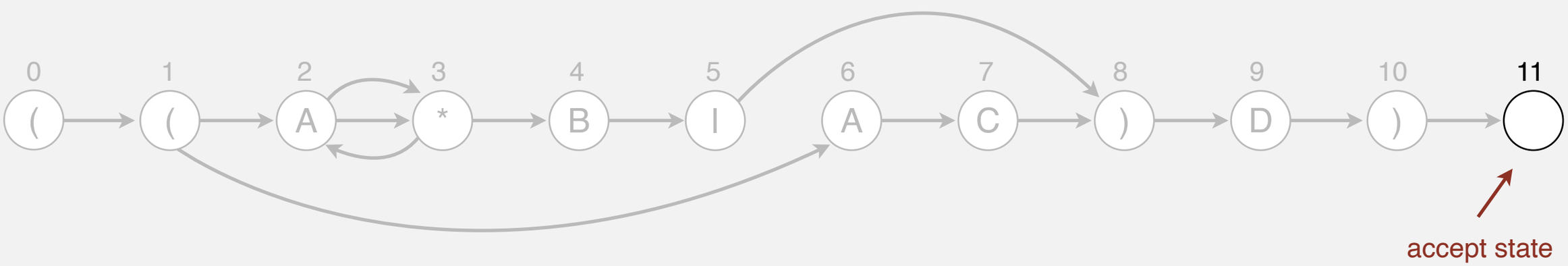
((A * B | A C) D)

NFA construction

End of regular expression.

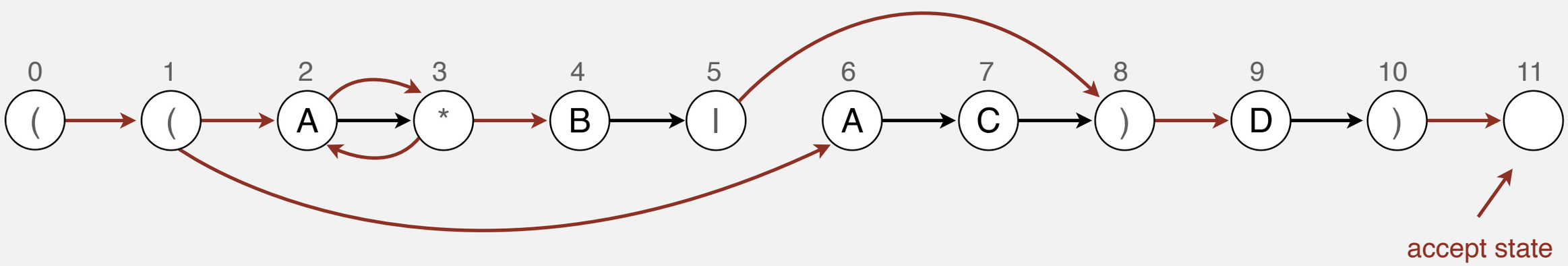
- Add accept state.





((A * B | A C) D)

NFA construction



NFA corresponding to the pattern ((A * B | A C) D)

NFA construction: Java implementation

```
private Digraph buildEpsilonTransitionDigraph() {
   Digraph G = new Digraph(M+1);
   Stack<Integer> ops = new Stack<Integer>();
   for (int i = 0; i < M; i++) {
      int lp = i;

      if (re[i] == '(' || re[i] == '|') ops.push(i);

      else if (re[i] == ')') {
         int or = ops.pop();
         if (re[or] == '|') {
            lp = ops.pop();
            G.addEdge(lp, or+1);
            G.addEdge(or, i);
         }
         else lp = or;
      }

      if (i < M-1 && re[i+1] == '*') {
         G.addEdge(lp, i+1);
         G.addEdge(i+1, lp);
      }

      if (re[i] == '(' || re[i] == '*' || re[i] == ')')
         G.addEdge(i, i+1);
   }
   return G;
}
```

left parentheses and |

or

closure

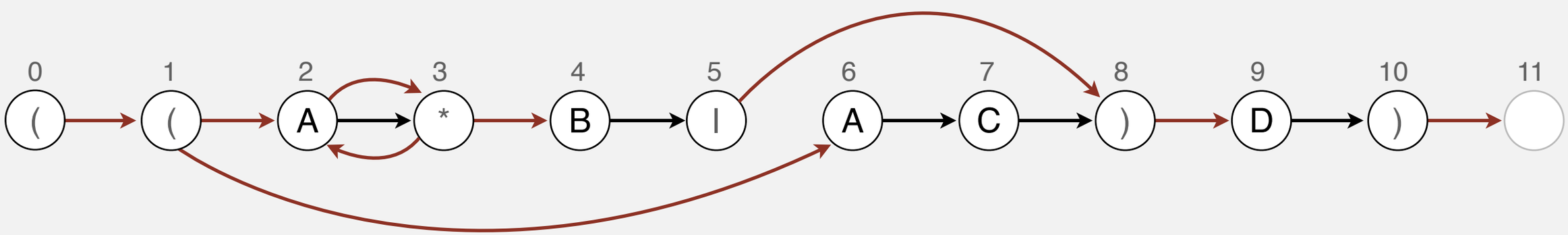
(needs 1-character lookahead)

metasymbols

NFA construction: analysis

Proposition. Building the NFA corresponding to an M-character RE takes time and space proportional to M.

Pf. For each of the M characters in the RE, we add at most three ε-transitions and execute at most two stack operations.



NFA corresponding to the pattern ((A * B | A C) D)

Regular Expressions

- REs and NFAs
- NFA simulation
- NFA construction
- **Applications**

Generalized regular expression print

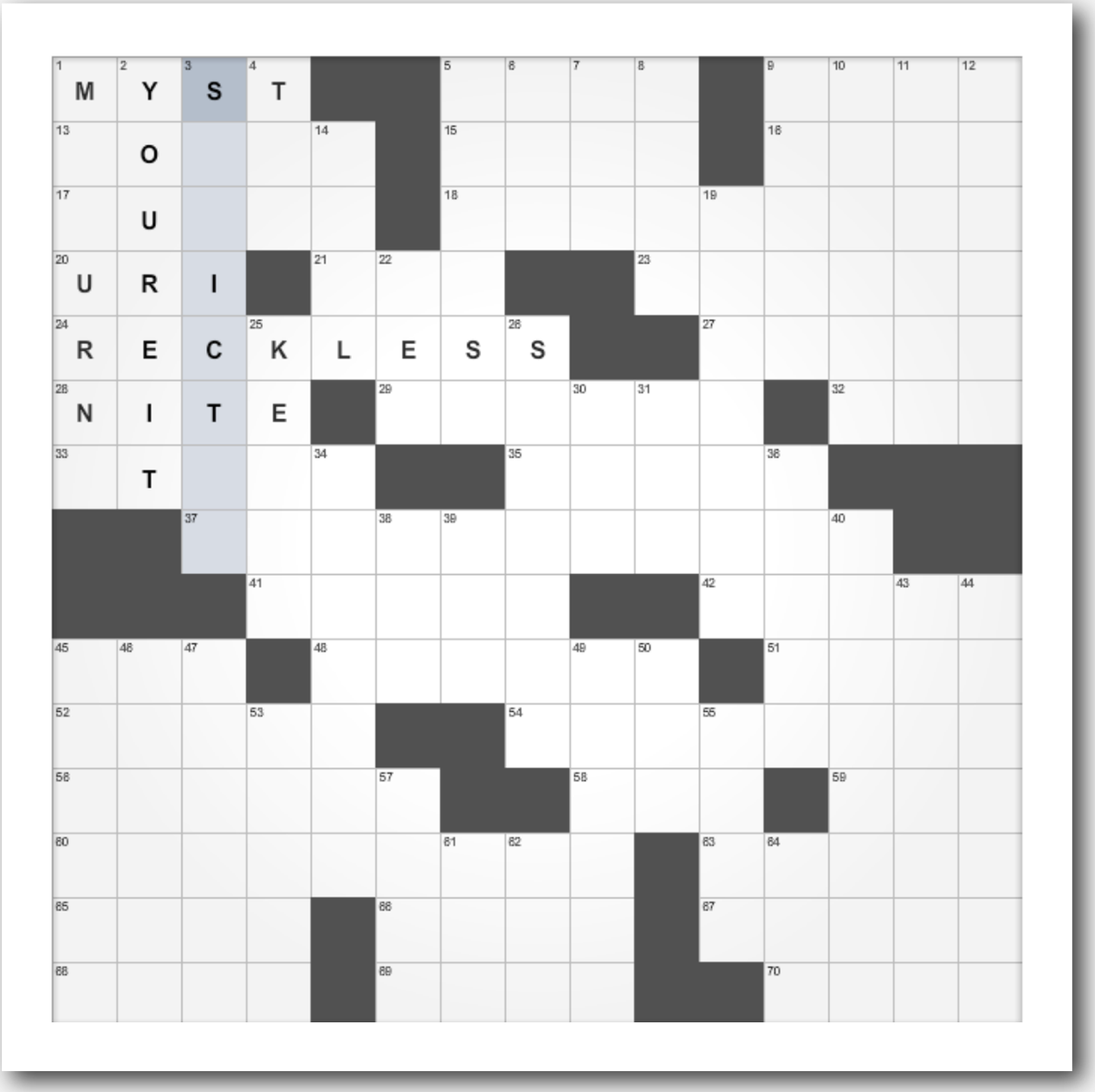
Grep. Take a RE as a command-line argument and print the lines from standard input having some substring that is matched by the RE.

```
public class GREP
{
   public static void main(String[] args)
   {
      String regexp = "(.*" + args[0] + ".*)";
      NFA nfa = new NFA(regexp);
      while (StdIn.hasNextLine())
      {
         String line = StdIn.readLine();
         if (nfa.recognizes(line))
            StdOut.println(line);
      }
   }
}
```

contains RE as a substring

Bottom line. Worst-case for grep (proportional to $M\,N$) is the same as for brute-force substring search.

Typical grep application: crossword puzzles



```
% more words.txt
a
aback
abacus
abalone
abandon
...

% grep "s..ict.." words.txt
constrictor
stricter
stricture
```

dictionary

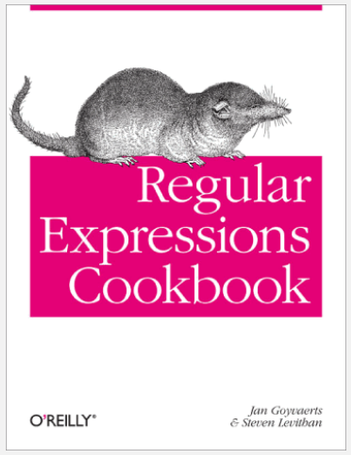
(standard in Unix)

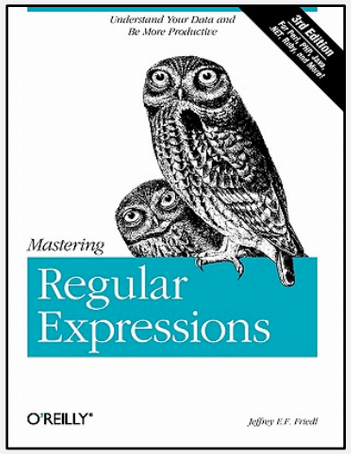
also on booksite

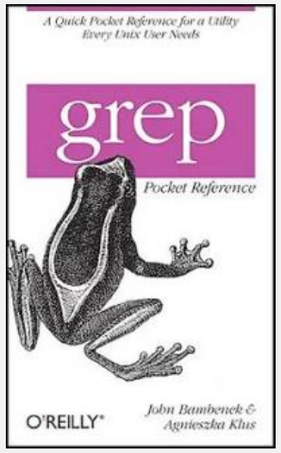
Industrial-strength grep implementation

To complete the implementation:

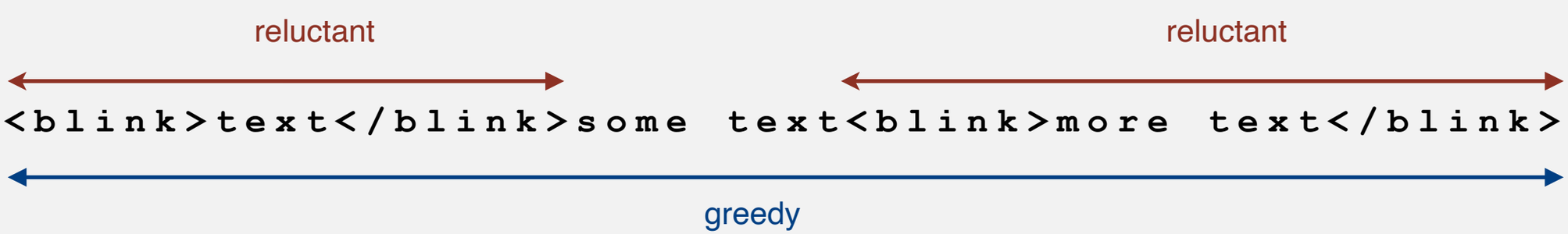
- Add character classes.
- Handle metacharacters.
- Add capturing capabilities.
- Extend the closure operator.
- Error checking and recovery.
- Greedy vs. reluctant matching.







Ex. Which substring(s) should be matched by the RE `<blink>.*</blink>` ?



Regular expressions in other languages

Broadly applicable programmer's tool.

- Originated in Unix in the 1970s.
- Many languages support extended regular expressions.
- Built into grep, awk, emacs, Perl, PHP, Python, JavaScript, ...

```
% grep 'NEWLINE' */*.java
```

print all lines containing **NEWLINE** which occurs in any file with a **.java** extension

```
% egrep '^[qwertyuiop]*[zxcvbnm]*$' words.txt | egrep '...........'
typewritten
```

PERL. Practical Extraction and Report Language.

```
% perl -p -i -e 's|from|to|g' input.txt
```

replace all occurrences of **from** with **to** in the file **input.txt**

```
% perl -n -e 'print if /^[A-Z][A-Za-z]*$/' words.txt
```

print all words that start with uppercase letter

do for each line

Regular expressions in Java

Validity checking. Does the `input` match the `regexp`?

Java string library. Use `input.matches(regexp)` for basic RE matching.

```
public class Validate
{
   public static void main(String[] args)
   {
      String regexp = args[0];
      String input  = args[1];
      StdOut.println(input.matches(regexp));
   }
}
```

```
% java Validate "[$_A-Za-z][$_A-Za-z0-9]*" ident123
true

% java Validate "[a-z]+@([a-z]+\.)+(edu|com)" rs@cs.princeton.edu
true

% java Validate "[0-9]{3}-[0-9]{2}-[0-9]{4}" 166-11-4433
true
```

legal Java identifier

valid email address (simplified)

Social Security number

Harvesting information

Goal. Print all substrings of input that match a RE.

```
% java Harvester "gcg(cgg|agg)*ctg" chromosomeX.txt
gcgcggcggcggcggcggctg
gcgctg
gcgctg
gcgcggcggcggaggcggaggcggctg
```

harvest patterns from DNA

harvest links from website

```
% java Harvester "http://(\\w+\\.)*(\\w+)" http://www.cs.princeton.edu
http://www.princeton.edu
http://www.google.com
http://www.cs.princeton.edu/news
```

Harvesting information

RE pattern matching is implemented in Java's `java.util.regexp.Pattern` and `java.util.regexp.Matcher` classes.

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class Harvester
{
   public static void main(String[] args)
   {
      String regexp   = args[0];
      In in           = new In(args[1]);
      String input    = in.readAll();
      Pattern pattern = Pattern.compile(regexp);
      Matcher matcher = pattern.matcher(input);
      while (matcher.find())
      {
         StdOut.println(matcher.group());
      }
   }
}
```

`compile()` creates a **`Pattern`** (NFA) from RE

`matcher()` creates a **`Matcher`** (NFA simulator) from NFA and text

`find()` looks for the next match

`group()` returns the substring most recently found by **`find()`**

Algorithmic complexity attacks

Warning. Typical implementations do not guarantee performance!

Unix grep, Java, Perl

```
% java Validate "(a|aa)*b" aaaaaaaaaaaaaaaaaaaaaaaaaaaaaac            1.6 seconds
% java Validate "(a|aa)*b" aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaac           3.7 seconds
% java Validate "(a|aa)*b" aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaac          9.7 seconds
% java Validate "(a|aa)*b" aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaac        23.2 seconds
% java Validate "(a|aa)*b" aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaac       62.2 seconds
% java Validate "(a|aa)*b" aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaac     161.6 seconds
```

SpamAssassin regular expression.

```
% java RE "[a-z]+@[a-z]+([a-z\.]+\.)+[a-z]+" spammer@x.....................
```

- Takes exponential time on pathological email addresses.
- Troublemaker can use such addresses to DOS a mail server.

Not-so-regular expressions

Back-references.

- `\1` notation matches subexpression that was matched earlier.
- Supported by typical RE implementations.

```
(.+)\1            // beriberi couscous
1?$|^(11+?)\1+    // 1111 111111 111111111
```

Some non-regular languages.

- Strings of the form ww for some string w: **`beriberi`**.
- Unary strings with a composite number of 1s: `111111`.
- Bitstrings with an equal number of 0s and 1s: `01110100`.
- Watson-Crick complemented palindromes: **`atttcggaaat`**.

Remark. Pattern matching with back-references is intractable.

Context

Abstract machines, languages, and nondeterminism.

- Basis of the theory of computation.
- Intensively studied since the 1930s.
- Basis of programming languages.

Compiler. A program that translates a program to machine code.

- KMP string $\Rightarrow$ DFA.
- `grep` RE $\Rightarrow$ NFA.
- `javac` Java language $\Rightarrow$ Java byte code.

	KMP	grep	Java
pattern	string	RE	program
parser	unnecessary	check if legal	check if legal
compiler output	DFA	NFA	byte code
simulator	DFA simulator	NFA simulator	JVM

Summary of pattern-matching algorithms

Programmer.

- Implement substring search via DFA simulation.
- Implement RE pattern matching via NFA simulation.

Theoretician.

- RE is a compact description of a set of strings.
- NFA is an abstract machine equivalent in power to RE.
- DFAs and REs have limitations.

You. Practical application of core computer science principles.

Example of essential paradigm in computer science.

- Build intermediate abstractions.
- Pick the right ones!
- Solve important practical problems.